

PYTHON

01

Fundamentals of Python Language

Write your first real programs — and understand exactly what Python is doing behind every line.

DURATION	LEVEL	MODULE	UPDATED
12 Sessions	Beginner	Module 1	May 2026

IN THIS MODULE

01	Introduction to Python
02	Variables & Data Types
03	Input, Output & Typecasting
04	Conditional Statements
05	Loops — for
06	Loops — while & Patterns
07	Break, Continue & Pass
08	Strings
09	Lists
10	Tuples
11	Dictionaries
12	Functions & Lambda

SESSION

01

60 MIN · FUNDAMENTALS

Introduction to Python

◆ Learning Objectives

- Explain what Python is and why it became the world's most popular programming language
- Identify at least four real-world use cases of Python across Indian industries
- Install Python 3 and VS Code on your machine successfully
- Write, save, and run your first Python script from VS Code
- Describe how Python reads and executes code line by line as an interpreted language

01 What is Python?

Python is a high-level, general-purpose programming language created by Guido van Rossum and first released in 1991. It is designed to be readable — the code looks close to plain English, so you can focus on solving the problem instead of fighting the language's syntax.

CONCEPT — High-Level Language

A high-level language hides complex machine-level details so you can focus entirely on your logic. When you write `print('Hello')` in Python, you are not managing memory addresses or CPU registers — Python takes care of all of that. Think of it like a UPI payment app: you just enter an amount and tap Pay; the banking network underneath is invisible to you.

PYTHON'S GLOBAL POPULARITY RANK

#1

Source: TIOBE Programming Community Index, May 2024

02 Why Python? Real-World Use Cases

Python is not just for theory. It runs real products you interact with every day. In India, companies like Zerodha (stock trading), Swiggy (logistics routing), and ISRO (satellite data processing) all rely on Python at their core.

Domain	What Python Does	Indian Example
Web Development	Builds websites and APIs using Django or Flask	Zerodha's trading platform backend

Domain	What Python Does	Indian Example
Data Science & AI	Analyses large datasets and trains ML models	ISRO satellite image processing
Automation & Scripting	Scripts that replace hours of manual, repetitive work	Auto-generating GST reports for SMEs in Surat
Finance & FinTech	Algorithmic trading, fraud detection, risk scoring	Paytm transaction risk engine
Education Tech	Adaptive learning platforms, quiz and recommendation systems	BYJU'S content recommendation engine

CASE STUDY — Zerodha & Python

Context — Zerodha is India's largest stock broker by active clients, headquartered in Bengaluru. It processes millions of trades every trading day.

Challenge — They needed to build real-time order matching, risk checks, and back-office reporting for millions of users — with a deliberately small engineering team to keep costs low.

What they did — Built their core trading engine and all internal tools almost entirely in Python, taking advantage of its large ecosystem of financial and data libraries and the ability to ship features quickly.

Result — Zerodha scaled to over 1.3 crore active clients and became profitable without the massive engineering headcount that traditional brokers required.

Takeaway — A small team with Python expertise can build and operate systems at a scale that would otherwise need ten times the people — which is exactly why the skill pays well.

Source: Zerodha Tech Blog — zerodha.com/z-connect, Nithin Kamath interviews (2022–23)

03 Installing Python & VS Code

You need two things installed on your machine: Python 3 (the language engine that runs your code) and VS Code (the editor where you write it). Both are completely free. Always download Python from the official site — python.org — to make sure you get the latest stable version.

WARNING — Tick the PATH checkbox

During the Windows Python installer, you must tick 'Add Python to PATH' before clicking Install Now. If you miss this step, your terminal will show 'python is not recognized as a command' and nothing will run. It is the #1 mistake beginners make — do not skip it.

```
python --version    Confirm Python is installed and print the version number
python3 --version   Use this on macOS or Linux if 'python' alone gives an error
pip --version       Confirm pip (Python's package installer) is also ready
```

TRY THIS LIVE

Open your terminal (Command Prompt on Windows, Terminal on macOS/Linux) and type: `python --version`. You should see 'Python 3.x.x'. If you see an error, raise your hand right now — fix it before moving on, because every session from here depends on this working.

04 Running Your First Python Script

There are two ways to run Python code: in the REPL (an interactive shell where you type one line at a time) or as a saved .py file. For real projects you will always use .py files. Write the classic first program, then customise it.

```
print('Hello, World!')
print('Kem cho, Ahmedabad!')
# This is a comment

print(2 + 2)
```

The classic first program — every programmer starts here
Python prints any language — it just outputs whatever text you give it
Lines starting with # are ignored by Python — use them to leave notes for yourself
Python can also evaluate expressions — this prints 4

TRY THIS LIVE

In VS Code, create a new file called `hello.py`. Type: `print('My name is [your name] and I am learning Python at TOPS!')`. Save the file (Ctrl+S) and press the green Run triangle at the top right. Your message should appear in the terminal at the bottom of the screen.

05 Python's Code Execution Model

Python is an interpreted language. When you press Run, Python reads your file from the very top, executes each line immediately, and then moves to the next. There is no separate 'compile' step before you can see results. If a line has an error, Python stops at exactly that line and tells you what went wrong.

Interpreted — Python

Reads and runs one line at a time. Errors appear instantly at the exact failing line. Fast feedback loop — ideal for learning, data work, and scripting. Slightly slower at runtime for heavy computation.

Compiled — C / Java

Translates the entire program into machine code first, then runs it. Faster at runtime but requires a separate build step before you can test anything. Error messages can be harder to trace for beginners.

“Python reads your code the way a teacher marks a paper — top to bottom, line by line, and it stops the moment it finds a mistake.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — Week 1, Lecture 1: Introduction to Python (Prof. Madhavan Mukund, IIT Madras)

Link — <https://nptel.ac.in/courses/106106145>

Always link the official channel — beware fan-made or re-upload copies.

WHERE TO GO NEXT

You have Python installed, your editor set up, and you understand how the language runs. This foundation is the same for every Python project you will ever build.

Explore deeper — Read the official Python tutorial intro at docs.python.org/3/tutorial/appetite.html — it is short, clear, and written by the language creators

Try on your own — Write a script that prints five things about yourself — your name, city, favourite food, a number, and a True/False statement about yourself

Connect the dots — Session 2 takes those five facts and teaches you exactly how Python stores each one in memory as a specific data type

Going further — Search 'CPython vs PyPy' if you are curious about how Python's execution model works at a deeper level under the hood

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python teacher for absolute beginners in India. Explain in simple Hinglish what an interpreted language means and how Python runs code line by line. Use an analogy from daily Indian life — like a dhaba cook making rotis one at a time rather than baking a whole batch first. Keep it under 150 words.

SESSION

02

60 MIN · FUNDAMENTALS

Variables & Data Types

◆ Learning Objectives

- Define what a variable is and explain how Python stores values in memory
- Create variables using correct Python naming rules and conventions
- Distinguish between the four primitive types: int, float, str, and bool
- Recognise the four non-primitive (collection) types: list, tuple, dict, and set
- Use `type()` to check a variable's data type and `id()` to inspect its memory address
- Build a complete student profile using multiple variable types in a single script

01 What is a Variable?

A variable is a named container that holds a value in your program's memory. Every time Python runs a line like `name = 'Raj'`, it reserves a space in memory, stores the value 'Raj' there, and labels that space with the name 'name'. After that, anywhere you write `name` in your code, Python fetches that value automatically.

CONCEPT — Variable as a Labelled Box

Imagine the memory of your computer as a row of storage lockers at a Gujarat GSRTC bus stand. A variable is like renting one locker and writing your name on it. You put something inside, and whenever you need it, you just say your name and the locker opens. You can also replace what is inside at any time — the label stays, the contents change.

<code>name = 'Riya'</code>	Creates a variable called 'name' and stores the text 'Riya' in it
<code>age = 20</code>	Creates 'age' and stores the whole number 20
<code>score = 87.5</code>	Creates 'score' and stores a decimal number
<code>is_enrolled = True</code>	Creates 'is_enrolled' and stores a True/False value
<code>print(name)</code>	Prints whatever value is currently inside 'name'

02 Naming Rules & Conventions

Python enforces a few hard rules for variable names, and experienced developers follow additional conventions to keep code readable. Breaking the hard rules causes a `SyntaxError`; ignoring the conventions just makes your code harder to maintain.

Rule / Convention	Correct Example	Wrong Example
Start with a letter or underscore	<code>student_name</code>	<code>1student</code> (starts with digit — Error)

Rule / Convention	Correct Example	Wrong Example
Use only letters, digits, underscores	<code>total_marks_2</code>	<code>total-marks</code> (hyphen not allowed — Error)
No spaces in names	<code>phone_number</code>	<code>phone number</code> (space not allowed — Error)
Case-sensitive — Name <code>name</code>	<code>firstName</code>	Using <code>FirstName</code> and <code>firstname</code> interchangeably
Convention: <code>snake_case</code> for variables	<code>monthly_salary</code>	<code>monthlySalary</code> (camelCase is valid but uncommon in Python)
Avoid reserved keywords	<code>user_input</code>	<code>input = 5</code> (shadows the built-in <code>input()</code> function)

PRO TIP — Name for the Reader, Not the Computer

Write variable names that a new person can understand instantly. `total_fees` is better than `tf`, and `student_age` is better than `x`. In your first job, your teammates will read your code every day — clear names save everyone time, including future-you.

03 Primitive Data Types

Python automatically assigns a data type to every value you store. The four primitive types are the building blocks — every more complex structure is built from combinations of these four.

Type	What It Stores	Example Value	Real-World Use
<code>int</code>	Whole numbers (no decimal)	<code>age = 21</code>	Student roll number, floor count in a building
<code>float</code>	Decimal numbers	<code>cgpa = 8.75</code>	Percentage marks, product price, GPS coordinates
<code>str</code>	Text — any characters in quotes	<code>city = 'Ahmedabad'</code>	Name, address, email, OTP message
<code>bool</code>	Only True or False	<code>is_active = True</code>	Is user logged in? Has payment cleared? Is seat available?

CONCEPT — Why Does the Type Matter?

Python behaves differently depending on the type. $10 + 5$ gives 15 (int addition), but `'10' + '5'` gives `'105'` (str joining). If your Aadhaar number is stored as an int, Python might strip the leading zero — but if stored as a str, it is preserved exactly. Choosing the right type from the start avoids bugs that are very hard to find later.

04 Non-Primitive (Collection) Types

Sometimes one variable needs to hold many values at once. Python provides four collection types for this. You will study each in depth in later sessions — for now, recognise them and understand when you would reach for each.

Type	Ordered?	Changeable?	Unique Values?	Example
list	Yes	Yes	No	<code>subjects = ['Python', 'SQL', 'Django']</code>
tuple	Yes	No	No	<code>coordinates = (23.02, 72.57)</code>
dict	Yes	Yes	Keys only	<code>student = {'name': 'Riya', 'age': 20}</code>
set	No	Yes	Yes	<code>unique_cities = {'Ahmedabad', 'Surat', 'Rajkot'}</code>

list []

An ordered, changeable sequence. Use when the order matters and you need to add or remove items later — like a to-do list or a list of enrolled students.

tuple ()

An ordered, unchangeable sequence. Use when the data should never be modified after creation — like GPS coordinates of a fixed location or a date of birth.

05 Inspecting Variables with `type()` and `id()`

Python gives you two built-in functions to look inside any variable. `type()` tells you what kind of data it holds. `id()` gives you its unique memory address — a number that proves Python has actually reserved a space in RAM for that value.

```
name = 'Riya'      Store a string value
print(type(name))  Output: <class 'str'> — confirms it is text
age = 20           Store a whole number
print(type(age))   Output: <class 'int'>
print(id(name))     Prints a large number — the memory address where 'Riya' is stored
```

```
age = 21
print(id(age))
```

Re-assign age to a new value
The id changes — Python created a new memory slot for 21

TRY THIS LIVE

Run this in VS Code: create two variables, `x = 5` and `y = 5`. Print `id(x)` and `id(y)`. Are the ids the same or different? Now try `x = 5` and `y = 6` and compare again. Python reuses memory for identical small integers — this is an optimisation called 'integer interning' and you have just discovered it yourself.

CASE STUDY — UIDAI Aadhaar & Data Types

Context — UIDAI (Unique Identification Authority of India) manages Aadhaar — the world's largest biometric database, covering over 135 crore residents. The system was largely architected and built in India, with significant Python usage in its analytics and verification layers.

Challenge — Each Aadhaar record holds many different kinds of information — a 12-digit ID number, a person's full name, date of birth, state, a boolean for biometric verification status, and fingerprint/iris data. Choosing the wrong data type for even one field causes failures at scale.

What they did — Engineers carefully modelled every field with its correct type: names as strings, the Aadhaar number itself as a string (not an integer, so leading zeroes are never lost), age as an integer, boolean flags for KYC status, and nested dictionaries for address data.

Result — The system handles over 50 lakh authentication requests per day reliably. Storing the Aadhaar number as a string — not an integer — was a small but critical data type decision that prevented data corruption across over a billion records.

Takeaway — The decision of which data type to use is not a detail — it is architecture. Getting it wrong at the start costs enormous effort to fix later.

Source: UIDAI Annual Report 2022–23, uidai.gov.in; public UIDAI tech architecture talks

◆ Putting It Together — Student Profile

```
student_name = 'Riya Shah'
roll_number  = '2024CS047'
age          = 20
cgpa         = 8.75
is_enrolled  = True
subjects     = ['Python', 'SQL', 'Django']

print(student_name, '|', type(student_name))
print(roll_number, '|', type(roll_number))
print(cgpa,      '|', type(cgpa))
```

str — full name
str — roll numbers can have letters, so never use int
int — whole years, no decimals needed
float — decimal precision matters for grades
bool — a yes/no fact
list — ordered, changeable collection of subjects

See the value and its type side by side

“In Python, every value has a type. Choosing the right type is the first real decision a programmer makes — and it matters from Day 1.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — Week 1, Lecture 2: Variables and Data Types (Prof. Madhavan Mukund, IIT Madras)

Link — <https://nptel.ac.in/courses/106106145>

WHERE TO GO NEXT

You can now create variables, choose the right type, and inspect them with `type()` and `id()`. These are the raw ingredients every Python program uses.

Explore deeper — Read 'Built-in Types' at docs.python.org/3/library/stdtypes.html — bookmark it, you will return to it often throughout the course

Try on your own — Build a shop item profile: create variables for a product name (str), price (float), quantity in stock (int), `is_available` (bool), and a list of three sizes. Print each with its type.

Connect the dots — Session 3 teaches `input()` and `print()` — you will learn how to let a user fill in that student profile from the keyboard instead of hardcoding the values yourself

Going further — Search 'Python dynamic typing vs static typing' to understand why Python does not force you to declare a type upfront — and what the trade-offs are

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor for a first-year student in India. Explain in simple Hinglish the difference between `int`, `float`, `str`, and `bool` with four everyday examples from Indian life — like an Aadhaar number, a train ticket price, a passenger name, and whether the train is on time. Keep it under 150 words.

SESSION
03

60 MIN · FUNDAMENTALS

Input, Output & Typecasting

◆ Learning Objectives

- Use `print()` to display text, numbers, and variables — including with `sep` and `end` options
- Collect keyboard input from a user using `input()`
- Explain why `input()` always returns a string — even when the user types a number
- Convert values between types using `int()`, `float()`, and `str()`
- Apply escape characters correctly inside string output
- Build a working four-operation basic calculator using `input` and typecasting

01 The `print()` Function — Sending Output

`print()` is Python's way of showing information to the user. You have already used it in Session 1. Now we look at what it can actually do — because it is more flexible than just printing one thing at a time.

```
print('Hello, World!')
print('Name:', 'Riya', 'City:', 'Surat')
print('Name:', 'Riya', sep=' | ')
print('Loading', end='...')
print('Done')
print(10, 20.5, True)
```

Basic print — one string

Multiple values separated by a space (default)

Custom separator — output: Name: | Riya

Replace the newline at the end with '...'

This now appears on the same line as 'Loading'

Python converts any type to text automatically for print

PRO TIP — f-strings Are the Modern Way

Instead of joining many values with commas, use an f-string: `name = 'Riya'` then `print(f'Hello, {name}!')`. The `f` before the quote tells Python to replace anything inside `{ }` with its value. f-strings are cleaner, faster, and what you will see in every real codebase from 2018 onwards.

```
name = 'Riya'
city = 'Surat'
cgpa = 8.75
print(f'Student: {name} from {city}, CGPA: {cgpa}')
```

Variable holding a name

Variable holding a city

Variable holding a decimal number

One clean line — no commas, no confusion

02 The input() Function — Collecting User Input

So far every value in your programs has been hardcoded by you. `input()` changes that — it pauses your program, shows a message to the user, waits for them to type something and press Enter, then hands that text to your program as a value.

<code>name = input('Enter your name: ')</code>	Shows prompt, waits, stores whatever the user types
<code>print(f'Hello, {name}!')</code>	Uses the collected name
<code>city = input('Which city are you from? ')</code>	You can collect as many inputs as you need
<code>print(f'{name} from {city}. Welcome!')</code>	Use both collected values together

WARNING — `input()` ALWAYS Returns a String

Even if the user types 25, Python stores it as the string '25', not the integer 25. This means `age + 1` will crash with a `TypeError` because you cannot add a number to text. You must convert the input before doing any arithmetic — which is exactly what Typecasting (Section 3) solves.

TRY THIS LIVE

Run this and observe the error: `age = input('Enter your age: ')` then `print(age + 1)`. Python will crash with `TypeError`. Now you understand exactly why typecasting exists — fix it in the next section.

03 Typecasting — Converting Between Data Types

Typecasting means explicitly converting a value from one data type to another. Python provides built-in functions for the most common conversions. You will use this constantly — virtually every program that takes user input needs it.

CONCEPT — Typecasting

Think of it like a currency exchange counter at Ahmedabad airport. You bring US Dollars (one type), the counter converts them to Indian Rupees (another type), and the amount in your hand changes form but represents the same value. `int('25')` takes the text '25' and gives you the number 25 that Python can actually do maths with.

Function	Converts To	Example	Output
<code>int(x)</code>	Whole number	<code>int('42')</code>	42
<code>int(x)</code>	Whole number	<code>int(8.9)</code>	8 (truncates — does NOT round)

Function	Converts To	Example	Output
float(x)	Decimal number	float('3.14')	3.14
float(x)	Decimal number	float(5)	5.0
str(x)	Text string	str(100)	'100'
str(x)	Text string	str(True)	'True'
bool(x)	True / False	bool(0)	False (0, "", [], None -> False; everything else -> True)

```
age_str = input('Enter your age: ')
age = int(age_str)
print('Next year you will be', age + 1)
```

input() gives us a string, e.g. '20'

Convert to integer so maths works

Now age + 1 works correctly — prints 21

```
price = float(input('Price in Rs : '))
gst = price * 0.18
print(f'Total with 18% GST: Rs {price + gst:.2f}')
```

:.2f inside f-string rounds to 2 decimal places

One-liner: convert immediately at the point of input

Now arithmetic works because price is a float

04 Escape Characters

Some characters have special meaning inside a Python string and cannot be typed directly. Escape characters are two-character codes — always starting with a backslash \ — that tell Python to treat the next character specially.

Escape Code	What It Does	Example Code	Output
\n	New line — moves to the next line	print('Line 1\nLine 2')	Line 1 (then) Line 2 on next line
\t	Tab — adds a horizontal indent	print('Name:\tRiya')	Name: Riya
\'	Single quote inside a single-quoted string	print('It\'s working!')	It's working!
\"	Double quote inside a double-quoted string	print("He said \"Hello\"")	He said "Hello"
\\	A literal backslash character	print('C:\\Users\\Riya')	C:\Users\Riya

PRO TIP — Raw Strings with r"

If your string has many backslashes (like a Windows file path), prefix it with r to turn off escape processing: `path = r'C:\Users\Riya\Documents'`. This is called a raw string and is widely used for file paths and regular expressions.

05 Putting It Together — Basic Calculator

This is the example from the lecture. It uses `input()` to collect two numbers, `float()` to typecast them, arithmetic to compute results, and f-strings with `\n` to format the output neatly. Every line connects to something from this session.

<code>print('=== TOPS Basic Calculator ===')</code>	Header printed once at the start
<code>num1 = float(input('Enter first number: '))</code>	Collect and immediately typecast to float
<code>num2 = float(input('Enter second number: '))</code>	Same for the second number
 <code>print(f'\nResults:')</code>	 <code>\n</code> adds a blank line before results
<code>print(f' Addition : {num1 + num2}')</code>	Works because both are float, not str
<code>print(f' Subtraction : {num1 - num2}')</code>	
<code>print(f' Multiplication: {num1 * num2}')</code>	
<code>print(f' Division : {num1 / num2:.4f}')</code>	<code>:.4f</code> formats the result to 4 decimal places

TRY THIS LIVE

Type the calculator above exactly as shown and run it. Then extend it: add a fifth line that prints the remainder (modulo) of `num1` divided by `num2` using the `%` operator. Test with `num1 = 17`, `num2 = 5` — the answer should be 2.

CASE STUDY — DigiGujarat Portal & Input Validation

Context — DigiGujarat (digitalgujarat.gov.in) is the Government of Gujarat's unified citizen services portal. It processes applications for dozens of schemes — from caste certificates to land records — submitted by lakhs of citizens across the state.

Challenge — Citizen submissions come entirely as text typed into web forms. Names, ages, incomes, mobile numbers, and Aadhaar numbers all arrive as strings. Using the wrong type for even one field — treating an income figure as a string instead of a number — causes eligibility calculations to fail silently.

What they did — The backend validation layer typecast every incoming field immediately on receipt: names stayed as strings, ages and income values were converted to integers and floats, mobile numbers were kept as strings (to preserve leading zeros), and boolean flags were derived from dropdown selections. Any value that failed typecasting was rejected with a specific error message before it could corrupt any downstream logic.

Result — The portal processed over 2.5 crore applications in 2022–23 with significantly reduced manual

re-work, because data type errors were caught at the entry point rather than discovered weeks later during processing.

Takeaway — `input()` gives you a string. The moment you decide what that string really represents — a number, a date, a flag — you must typecast it. Doing this at the entry point is the professional habit.

Source: *DigiGujarat Annual Report 2022–23*, digitalgujarat.gov.in

“input() is the bridge between the human world and the code world. Typecasting is the toll gate — you pay it once, correctly, and everything after that runs smoothly.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — Week 1, Lecture 3: Input and Output in Python (Prof. Madhavan Mukund, IIT Madras)

Link — <https://nptel.ac.in/courses/106106145>

WHERE TO GO NEXT

You can now collect input, output formatted results, and safely convert between types — your programs can talk to a real user for the first time.

Explore deeper — Read about Python's built-in input/output functions at docs.python.org/3/library/functions.html#input and docs.python.org/3/library/functions.html#print

Try on your own — Build a 'GST Calculator': ask the user for a product price and a GST percentage, compute the GST amount and the total price, and display all three values formatted to 2 decimal places

Connect the dots — Session 4 introduces conditional statements — you will use `if/else` to make your programs respond differently depending on what the user inputs

Going further — Search 'Python format specification mini-language' to learn all the ways you can control how numbers and text are displayed inside f-strings

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor for a beginner student in India. Explain in simple Hinglish why `input()` always returns a string and why typecasting is needed before doing maths. Use an analogy involving a shopkeeper at a kiriyana store who only accepts written slips — and the slip always says words, never numbers. Keep it under 150 words.

SESSION

04

60 MIN · FUNDAMENTALS

Conditional Statements

◆ Learning Objectives

- Explain what a conditional statement is and why programs need decision-making logic
- Write if and if-else statements that execute different blocks based on a condition
- Chain multiple conditions using elif to handle more than two possible outcomes
- Use nested if statements to check secondary conditions inside a primary one
- Combine conditions using and, or, and not logical operators
- Build a grade calculator and a blood donation eligibility checker from scratch

01 What is a Conditional Statement?

Every useful program makes decisions. Should this user see the admin panel or the student panel? Does this person qualify for a loan? Is the seat available? Conditional statements are how Python evaluates a situation and chooses which block of code to run.

CONCEPT — Condition & Boolean Result

Every condition Python checks is a question that has exactly two answers: True or False. `age >= 18` is either True (the person is 18 or older) or False (they are not). Python evaluates the condition, gets a boolean result, and then decides which path to take — exactly like an eligibility checker at a Gujarat government scheme desk that first checks your age before processing your application.

Operator	Meaning	Example	Result
<code>==</code>	Equal to	<code>5 == 5</code>	True
<code>!=</code>	Not equal to	<code>5 != 3</code>	True
<code>></code>	Greater than	<code>10 > 7</code>	True
<code><</code>	Less than	<code>3 < 1</code>	False
<code>>=</code>	Greater than or equal to	<code>18 >= 18</code>	True
<code><=</code>	Less than or equal to	<code>20 <= 19</code>	False

02 if and else — Two Paths

The simplest decision is a fork in the road: if the condition is True, do this; otherwise, do that. In Python, the indented block under if runs when the condition is True. The indented block under else runs when it is False. Indentation is not optional — it is how Python knows which lines belong to which branch.

<code>age = int(input('Enter your age: '))</code>	Collect and typecast user input
<code>if age >= 18:</code>	Condition — is the person 18 or older?
<code>print('You can vote.')</code>	This runs ONLY if condition is True — note the 4-space indent
<code>else:</code>	This block runs if the condition was False
<code>print('You cannot vote yet.')</code>	Indented under else — part of the else block

WARNING — Indentation Errors Are Real Errors

Python uses indentation (spaces) to define code blocks — there are no curly braces { } like in Java or C. If your indented block has inconsistent spaces (mixing tabs and spaces, or using 3 spaces instead of 4), Python throws an `IndentationError` and your program will not run. VS Code's Python extension handles this automatically — let it, and never mix tabs with spaces manually.

03 elif — Multiple Branches

When there are more than two possible outcomes, use `elif` (short for 'else if'). Python checks each condition in order from top to bottom and runs the first block whose condition is True. Once a match is found, all remaining `elif` and `else` branches are skipped entirely.

<code>marks = int(input('Enter your marks (0-100): '))</code>	
Collect marks from the user	
 <code>if marks >= 90:</code>	Checked first
<code>grade = 'A+'</code>	Runs if marks is 90 or above
<code>elif marks >= 75:</code>	Only checked if first condition was False
<code>grade = 'A'</code>	Runs if marks is 75–89
<code>elif marks >= 60:</code>	
<code>grade = 'B'</code>	Runs if marks is 60–74
<code>elif marks >= 45:</code>	
<code>grade = 'C'</code>	Runs if marks is 45–59
<code>else:</code>	Catches everything below 45
<code>grade = 'Fail'</code>	
 <code>print(f'Your grade: {grade}')</code>	Printed after all conditions are evaluated

TRY THIS LIVE

Run the grade calculator above. Test it with at least five inputs: 95, 80, 62, 50, and 30. Confirm each one gives the correct grade. Then add a new band: if marks ≥ 35 and marks < 45 , grade should be 'D' instead of 'Fail'. Insert the correct elif line in the right position.

04 Logical Operators — and, or, not

Real eligibility checks rarely depend on just one condition. A home loan requires the applicant to be over 21 AND have a salary above a certain amount. A student gets a discount if they have a college ID OR a school ID. Python's logical operators let you combine multiple conditions into one.

Operator	Returns True When	Example
and	Both conditions are True	age ≥ 18 and salary ≥ 25000
or	At least one condition is True	has_college_id or has_school_id
not	The condition is False (inverts)	not is_banned

```
# Blood Donation Eligibility Checker
age = int(input('Enter your age: '))
weight = float(input('Enter your weight (kg): '))
```

From the lecture flow — uses and + nested if

```
if age >= 18 and age <= 65:
    if weight >= 50:
        print('Eligible to donate blood.')
    else:
```

Primary check: age must be in range

Secondary check — only runs if age passed

Both conditions met

```
print('Weight too low. Minimum: 50 kg.')
    Age OK, weight not OK
```

```
else:
    print('Age not in eligible range (18-65).')
```

Primary check failed — no need to check weight

05 Nested if — Conditions Inside Conditions

A nested if is simply an if block placed inside another if block. Use nesting when the second condition only makes sense to check after the first has already passed. In the blood donation example above, checking weight is only meaningful after confirming the age is valid — which is why the weight check is nested inside the age check.

CONCEPT — When to Nest vs When to Use 'and'

If both conditions must be True to do the same thing, use and in a single if: if age >= 18 and weight >= 50. If each failed condition needs a different error message, use nested if — so you can tell the user exactly which requirement they missed. The blood donation example uses nesting because 'wrong age' and 'too light' deserve different feedback.

and — one line, one message

```
if age >= 18 and weight >= 50:
    print('Eligible')
else:
    print('Not eligible')
```

Simple. But the else cannot tell the user whether they failed the age check or the weight check.

Nested if — specific feedback

```
if age >= 18 and age <= 65:
    if weight >= 50:
        print('Eligible')
    else:
        print('Weight too low')
else:
    print('Age out of range')
```

More lines, but the user knows exactly what to fix.

TRY THIS LIVE

Extend the blood donation checker: add a third condition — the donor must not have donated in the last 90 days. Add a variable `days_since_last = int(input('Days since last donation: '))`. Nest this check inside the weight check, and print a specific message if the gap is too short.

CASE STUDY — IRCTC Tatkal Booking Rules

Context — IRCTC (Indian Railway Catering and Tourism Corporation) handles online train ticket booking for over 13 lakh passengers every single day. Tatkal tickets have strict eligibility windows and pricing rules.

Challenge — Tatkal booking opens exactly 24 hours before departure for AC classes and 24 hours for Sleeper — but not before 10:00 AM on the day of booking. Booking must be blocked for the first 30 minutes for senior citizens. Prices change based on travel class, distance, and time of booking. All of this has to be enforced automatically across millions of concurrent requests.

What they did — Every booking attempt passes through a chain of conditional checks in order: Is the booking window open? Is the user a senior citizen trying within the first 30 minutes? Does the fare match the distance and class rules? Only after all conditions pass does the seat get offered. Each failed condition produces a specific, different error message to the user.

Result — IRCTC processes over 7 lakh Tatkal bookings per day during peak season with automated rule enforcement — no human needs to manually verify each ticket's eligibility.

Takeaway — Conditional logic is not just a programming concept — it is the engine behind every rule-based system, from train booking to loan approvals to exam eligibility.

Source: IRCTC Annual Report 2022–23; Ministry of Railways press releases

“Code without conditions can only do one thing. Code with conditions can make decisions — and that is what separates a calculator from a real program.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — Week 2, Lecture 1: Conditional Statements (Prof. Madhavan Mukund, IIT Madras)

Link — <https://nptel.ac.in/courses/106106145>

WHERE TO GO NEXT

Your programs can now make decisions. Every real application — login systems, eligibility checks, dashboards — is built on conditional logic exactly like this.

Explore deeper — Read 'More Control Flow Tools' at docs.python.org/3/tutorial/controlflow.html — the official Python tutorial on if statements and related structures

Try on your own — Build a 'Loan Eligibility Checker': ask for age (must be 21–60), monthly salary (must be Rs 20,000), and CIBIL score (must be 700). Print a specific rejection reason if any one condition fails, or 'Eligible for loan' if all three pass

Connect the dots — Session 5 introduces for loops — the next big control flow tool. Instead of choosing between paths, loops let you repeat a block of code many times automatically

Going further — Search 'Python ternary operator' to learn how to write a simple if-else on a single line: `value = 'Pass' if marks >= 45 else 'Fail'`

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor for a beginner student in India. Explain in simple Hinglish how if, elif, and else work together using the example of a school grade system — marks above 90 get A+, 75–90 get A, 60–75 get B, and below 60 is Fail. Show the Python code and explain each line in Hinglish. Keep it under 200 words.

SESSION

05

60 MIN · FUNDAMENTALS

Loops — for

◆ Learning Objectives

- Explain what a loop is and why repetition is one of the most powerful ideas in programming
- Write a for loop that iterates over a list, a string, and a range of numbers
- Use range(start, stop, step) to control exactly which numbers a loop visits
- Loop through a list of items and process each one individually
- Count characters in a string using a for loop
- Combine loops with conditional statements to filter or transform data

01 Why Do We Need Loops?

Imagine you need to print the names of 500 students. Without a loop, you would write 500 `print()` statements. With a loop, you write two lines. Loops let your program repeat an action automatically — once for each item in a collection, or a specific number of times. This is where Python starts to feel genuinely powerful.

CONCEPT — Iteration

Iteration means going through a collection one item at a time and doing something with each item. Think of a post office worker in Ahmedabad sorting the morning's letters: they pick up each envelope, read the address, place it in the right tray, then move to the next. The task is the same every time — only the envelope changes. A for loop does exactly this with data.

LINES OF CODE SAVED

498

Printing 500 names with a loop takes 2 lines. Without a loop: 500 lines.

02 The for Loop — Syntax

The for loop in Python reads almost like English: 'for each item in this collection, do the following'. The loop variable (the word after for) automatically takes the value of the current item on every pass through the loop.

```
fruits = ['Mango', 'Banana', 'Apple', 'Papaya', 'Guava']
```

A list of five fruit names

```
for fruit in fruits:
```

On each pass, 'fruit' becomes the next item in the list

```
print(fruit)
```

Indented — this line runs once per item. Prints 5 times total.

CONCEPT — The Loop Variable

The loop variable (fruit in the example above) is just a regular variable that Python automatically updates on each pass. You choose the name — pick something meaningful. `for item in fruits` is valid but `for fruit in fruits` reads like a sentence and is far clearer. Good loop variable names make for loops nearly self-documenting.

```
students = ['Riya', 'Arjun', 'Meera', 'Dev', 'Pooja']
```

A list of student names

```
for student in students:
```

```
    print(f'Welcome to TOPS, {student}!')
```

Loop variable is 'student' — singular of 'students'

Runs 5 times, each time with a different name

TRY THIS LIVE

Create a list called `cities` with five Gujarat city names. Write a `for` loop that prints each city in the format: 'City: [name]'. Then add a second line inside the loop that also prints the number of characters in each city name using `len(city)`. Both lines inside the loop will run for every city.

03 Looping Through Strings

A string is a sequence of characters, and `for` loops work on any sequence — not just lists. When you loop over a string, the loop variable takes the value of each individual character one at a time.

```
city = 'Ahmedabad'
```

A string with 9 characters

```
for char in city:
```

Each pass: `char = 'A'`, then `'h'`, then `'m'`, etc.

```
    print(char)
```

Prints one character per line — 9 lines total

◆ Counting Characters — A Classic Loop Example

```
word = input('Enter a word: ')
```

Collect any word from the user

```
count = 0
```

Start the counter at zero before the loop

```
for char in word:
```

Visit each character one by one

```
    count = count + 1
```

Add 1 to count on every pass

```
print(f'Characters: {count}')
```

After the loop finishes, print the total

```
# Shortcut — but understand the loop version first:
```

```
print(len(word))
```

len() does the same job — now you know what it is doing internally

PRO TIP — Always Initialise Counters Before the Loop

The line `count = 0` must appear before the for loop, not inside it. If it were inside, Python would reset count back to zero on every single pass and you would always end up with 1. Initialise outside, update inside — this pattern applies to counters, totals, and any accumulating variable.

04 The range() Function

Sometimes you want to loop a specific number of times, or step through a sequence of numbers, rather than iterate over an existing list. `range()` generates that sequence of numbers on the fly without creating a list in memory.

range() Call	Numbers Produced	Common Use
<code>range(5)</code>	0, 1, 2, 3, 4	Loop exactly 5 times (zero-indexed)
<code>range(1, 6)</code>	1, 2, 3, 4, 5	Loop from 1 to 5 inclusive
<code>range(1, 11)</code>	1, 2, 3, ... 10	Loop 10 times with human-friendly numbering
<code>range(0, 10, 2)</code>	0, 2, 4, 6, 8	Even numbers — step of 2
<code>range(10, 0, -1)</code>	10, 9, 8, ... 1	Count down from 10 to 1

```
# Print table of 5
for i in range(1, 11):
    print(f'5 x {i} = {5 * i}')
```

A classic school example done the programmer's way
i goes from 1 to 10 (stop=11 is excluded)
Prints the full multiplication line

```
# Print roll numbers 1 to 30
for roll in range(1, 31):
    print(f'Roll No. {roll}')
```

Generates 1, 2, 3 ... 30
Prints all 30 — with 2 lines of code

05 Combining Loops with Conditions

A for loop and an if statement can be combined — the condition runs on every pass of the loop. This lets you filter, flag, or separate items as you go through a collection, which is one of the most common patterns in real code.

```
marks_list = [82, 45, 91, 33, 67, 78, 29, 88]
```

List of eight students' marks

```
for marks in marks_list:
    if marks >= 45:
        print(f'{marks} – Pass')
    else:
        print(f'{marks} – Fail')
```

Loop visits each score one at a time
 Condition checked on every pass
 Runs only when marks is 45 or above
 Runs when marks is below 45

TRY THIS LIVE

Using the marks_list above, write a loop that counts how many students passed (marks >= 45) and how many failed. Initialise two counters before the loop: pass_count = 0 and fail_count = 0. Inside the loop, increment the right counter based on the condition. After the loop, print both totals.

CASE STUDY — Amul & Daily Milk Procurement Processing

Context — Amul (Gujarat Co-operative Milk Marketing Federation), headquartered in Anand, Gujarat, is the world's largest farmer-owned dairy cooperative. It collects milk from over 36 lakh farmers across 18,600+ village dairy co-operative societies every single day.

Challenge — Each morning, every collection centre records the quantity and fat content of milk from each farmer. By end of day, all 36 lakh records must be processed to calculate payment amounts, flag below-standard batches, and generate district-level summaries — accurately and automatically.

What they did — The procurement software loops through every farmer record collected that day. For each record: it reads the quantity and fat percentage, applies a conditional check (does fat content meet the minimum standard?), calculates the payment rate based on fat content bracket, and accumulates district-level totals. The entire daily batch runs as a loop over millions of records with nested conditions inside.

Result — Farmers across Gujarat receive accurate SMS payment notifications within hours of the evening collection — a process that would take thousands of manual workers days to complete by hand.

Takeaway — A for loop with conditions inside is not a classroom exercise. It is the engine behind every batch processing system — from milk procurement to payroll to logistics.

Source: Amul Annual Report 2022–23, [amul.com](https://www.amul.com); GCMMF public technology briefings

“A loop lets one line of code do the work of a thousand. That is not a shortcut — it is the whole point of programming.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — Week 2, Lecture 2: Loops and Iteration (Prof. Madhavan Mukund, IIT Madras)

Link — <https://nptel.ac.in/courses/106106145>

WHERE TO GO NEXT

You can now repeat actions across any collection of data automatically. Every real system — from payroll to report generation — is built on this pattern.

Explore deeper — Read about for loops and range() at

docs.python.org/3/tutorial/controlflow.html#for-statements — pay attention to the note about modifying lists while looping

Try on your own — Build a 'Marks Analyser': create a list of 10 student marks. Loop through it to count passes and fails, find the highest mark, and calculate the class average — all in one loop pass

Connect the dots — Session 6 introduces the while loop — a different kind of loop that keeps running until a condition becomes False, rather than until a collection runs out

Going further — Search 'Python enumerate()' to learn how to get both the index and the value of each item during a for loop — used constantly in real code

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor for a beginner student in India. Explain in simple Hinglish what a for loop does and how range() works. Use the real-life example of a postman delivering letters to houses numbered 1 to 50 — visiting each house in order and doing the same task at each door. Show the Python code equivalent. Keep it under 200 words.

SESSION

06

60 MIN · FUNDAMENTALS

Loops — while & Patterns

◆ Learning Objectives

- Explain how a while loop differs from a for loop and when to choose each
- Write a while loop that runs as long as a condition remains True
- Identify an infinite loop and explain how to prevent one
- Use a while loop to handle real-world retry logic with a counter
- Build right-triangle and square star patterns using while loops

01 The while Loop — Run Until False

A for loop runs once for each item in a known collection. A while loop keeps running as long as a condition stays True — and it has no idea in advance how many times that will be. Use while when the number of repetitions depends on something that changes during the run, like user input or a countdown.

CONCEPT — while vs for

Think of a for loop like a GSRTC bus with fixed stops — it visits every stop on the route exactly once and then finishes. A while loop is like waiting at a dhaba for a table — you keep waiting as long as all tables are full. The moment a table frees up (condition becomes False), you stop waiting and act. You do not know in advance how many minutes that will take.

for — known count

Use when you know what you are iterating over: a list, a string, or a fixed range of numbers.

```
for student in students:
    print(student)
```

Python knows when to stop because the list has a fixed length.

while — unknown count

Use when the stop condition depends on something that changes at runtime: user input, a score crossing a threshold, a retry limit.

```
while attempts < 3:
    pin = input('Enter PIN: ')
```

Python keeps going until attempts reaches 3.

```
count = 1
while count <= 5:
    print(f'Count: {count}')
    count = count + 1
print('Loop finished.')
```

Initialise the loop variable BEFORE the while loop
Condition checked before every pass — if False, loop ends immediately
Indented block — runs only when condition is True
CRITICAL: update the variable — without this, loop runs forever
This line is NOT indented — it runs after the loop ends

02 Infinite Loops — and How to Avoid Them

An infinite loop is a while loop whose condition never becomes False. The program hangs, the terminal freezes, and nothing else runs. Every beginner creates one by accident. Knowing the cause makes it easy to prevent.

WARNING — The Most Common Infinite Loop Mistake

Forgetting to update the variable that the condition depends on. If count starts at 1 and you check while count <= 5, but never write count = count + 1 inside the loop, count stays at 1 forever and the condition is always True. Your program will run until you force-stop it with Ctrl + C in the terminal.

<pre># WRONG — infinite loop count = 1 while count <= 5: print('This prints forever!')</pre>	Do not run this as-is count never changes, so this is always True Runs endlessly — Ctrl+C to stop
<pre># CORRECT — always update the control variable count = 1 while count <= 5: print(count) count += 1</pre>	count += 1 is shorthand for count = count + 1

PRO TIP — while True with a Break

Experienced Python developers sometimes write while True: deliberately — an intentional loop that runs forever until a break statement exits it from inside. This pattern is common for menus, login screens, and retry systems. You will see break in Session 7. For now, just know that while True is a valid and common pattern, not always a bug.

03 Retry Logic — A Real-World while Pattern

The most common real-world use of a while loop is retry logic — keep trying until something succeeds or you hit a maximum number of attempts. Every login screen, PIN entry system, and OTP verifier you have ever used is built on exactly this pattern.

<pre>correct_pin = '1234'</pre>	In a real system this would come from a secure database
<pre>attempts = 0</pre>	Counter starts at 0 before the loop
<pre>max_attempts = 3</pre>	Maximum tries allowed
<pre>while attempts < max_attempts:</pre>	Keep going while attempts is under the limit

```

pin = input('Enter your PIN: ')
if pin == correct_pin:
    print('Access granted. Welcome!')
    break

else:
    attempts += 1
    remaining = max_attempts - attempts
print(f'Wrong PIN. {remaining} attempt(s) left.')

if attempts == max_attempts:
    print('Card blocked. Contact your bank.')

```

Ask user for PIN on every pass

Check if it matches

Exit the loop — no more tries needed (covered fully in Session 7)

Wrong PIN — increment the attempt counter

Loop ended without a break — all attempts used up

CASE STUDY — SBI ATM PIN Retry System

Context — State Bank of India operates the largest ATM network in India, with over 65,000 ATMs serving crores of customers across every state including 1,200+ ATMs in Gujarat alone.

Challenge — ATM systems need to allow legitimate users to retry a mistyped PIN while blocking attackers who try to guess it. Allowing unlimited retries makes guessing easy; blocking after one wrong try frustrates genuine customers.

What they did — A while loop runs on every PIN entry attempt: check if the entered PIN matches, increment an attempt counter if not, and block the card permanently after 3 failures. The loop exits cleanly on success (correct PIN) or on exhaustion (3 wrong tries). This logic runs billions of times a day across all Indian bank ATMs.

Result — The 3-attempt policy became an RBI-mandated standard across all Indian banks. The while loop pattern behind it balances customer convenience against fraud prevention at national scale.

Takeaway — The while loop you write to validate a PIN in class is structurally identical to what runs in every ATM in India. The scale changes — the pattern does not.

Source: RBI ATM Security Guidelines 2021; SBI Annual Report 2022–23, [sbi.co.in](https://www.sbi.co.in)

04 Star Patterns with while Loops

Printing star patterns is the classic exercise for building intuition about how loop variables change over time. The key insight: the number of stars on each row equals the row number — which is the same as the current value of your loop variable.

◆ Pattern 1 — Right-Angled Triangle

```

rows = 5
i = 1

```

Total number of rows in the pattern

Start from row 1

```
while i <= rows:
    print('*' * i)
    i += 1
```

Run once for each row
 Print i stars — row 1 gets 1 star, row 2 gets 2, etc.
 Move to the next row

◆ Pattern 2 — Inverted Triangle

```
rows = 5
i = rows
while i >= 1:
    print('*' * i)
    i -= 1
```

Start from maximum width
 i starts at 5 and counts down
 Run until i reaches 1
 Row 1 gets 5 stars, row 2 gets 4, etc.
 Count DOWN instead of up

TRY THIS LIVE

After running both patterns above, try to build a full diamond: combine the right-angled triangle and the inverted triangle back to back in a single script. The first half counts up from 1 to 5 stars, and the second half counts back down from 4 to 1. Hint: you need two separate while loops.

“A for loop asks ‘what is next in the list?’ A while loop asks ‘is the situation still the same?’ Both questions are essential — knowing which to ask is the skill.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — Week 2, Lecture 3: While Loops (Prof. Madhavan Mukund, IIT Madras)

Link — <https://nptel.ac.in/courses/106106145>

WHERE TO GO NEXT

You now have both loop types. The next step is learning how to control them mid-run — exiting early, skipping items, and leaving placeholders.

Explore deeper — Read Python's official while loop documentation at docs.python.org/3/reference/compound_stmts.html#the-while-statement

Try on your own — Build a number-guessing game: generate a secret number between 1 and 10 (use the random module — `random.randint(1, 10)`), then use a while loop to keep asking the user to guess until they get it right. Count and display the number of attempts at the end.

Connect the dots — Session 7 introduces break, continue, and pass — the three statements that give you fine-grained control over what happens inside any loop, whether for or while

Going further — Search 'Python do-while equivalent' — Python has no built-in do-while loop. Find out how Python programmers simulate one using while True and break

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor for a beginner student in India. Explain in simple Hinglish the difference between a for loop and a while loop. Use the analogy of a GSRTC bus route (for — fixed stops) versus waiting in an OTP verification screen (while — keep going until the right code is entered). Keep it under 200 words.

SESSION
07

60 MIN · FUNDAMENTALS

Break, Continue & Pass

◆ Learning Objectives

- Explain why loop control statements exist and what problem each one solves
- Use `break` to exit a loop immediately when a target condition is met
- Use `continue` to skip the current iteration and move to the next one
- Use `pass` as a valid placeholder that lets Python parse an empty block without crashing
- Combine `break` and `continue` with both `for` and `while` loops in realistic scenarios

01 Why Loop Control Statements Exist

A basic loop runs through every item from start to finish. But real problems rarely need that. You might want to stop the moment you find what you are looking for, skip items that do not qualify, or leave a block of code empty for now while the rest of the program still runs. Python provides three statements for exactly these situations: `break`, `continue`, and `pass`.

Statement	What It Does	When to Use It
<code>break</code>	Exits the loop immediately — no more iterations	You found what you need and further looping is wasteful
<code>continue</code>	Skips the rest of this iteration and jumps to the next one	This item does not qualify — process the rest without it
<code>pass</code>	Does nothing — acts as a placeholder for empty blocks	You want to write the structure now and fill the logic later

02 `break` — Exit the Loop Early

`break` tells Python to exit the loop immediately, regardless of how many items or iterations remain. Execution jumps to the first line after the loop block. If there is an `else` clause on the loop, it is skipped when `break` is used.

CONCEPT — `break`

Think of a security guard at a GIFT City office building checking visitor IDs against a watchlist. The moment they find a match — they stop checking the rest of the list and act. Continuing to scan all remaining IDs after a match is found is wasteful and slow. `break` is the instruction to stop as soon as the mission is complete.

```
# Lecture example: Break at 'Banana'
```

```
fruits = ['Apple', 'Mango', 'Banana', 'Papaya', 'Guava']
```

List of five fruits

```
for fruit in fruits:
```

```
    if fruit == 'Banana':
```

```
        print('Found Banana — stopping.')
```

```
        break
```

```
print(fruit)
```

Loop visits each fruit in order

Check if current item is 'Banana'

Announce the find

Exit the loop NOW — Papaya and Guava are never visited

Only runs if break was NOT triggered — prints Apple, Mango

◆ break with a while Loop

```
while True:
```

```
    response = input('Type quit to exit: ')
```

```
    if response == 'quit':
```

```
        print('Goodbye!')
```

```
        break
```

```
    print(f'You typed: {response}')
```

Intentional infinite loop — relies on break to exit

Ask user on every pass

Only exit point — loop runs forever until this triggers

Runs on every pass where user did not type quit

TRY THIS LIVE

Create a list of numbers: `numbers = [4, 17, 3, 29, 8, 41, 15]`. Write a for loop that searches for the first number greater than 20. Use break when you find it and print: 'First number over 20: [number]'. If no number qualifies, nothing should print after the loop. Test by changing the list so no number exceeds 20 — confirm nothing prints.

03 continue — Skip This Item, Keep Going

continue skips the rest of the current iteration's code block and immediately moves to the next item in the loop. Unlike break, it does not exit the loop — all remaining items are still processed.

CONCEPT — continue

Think of a Swiggy delivery agent sorting through a batch of orders. If an order is marked 'cancelled', they skip it and move to the next one — they do not stop work for the day (that would be break). continue is the 'skip this one, move on' instruction — the loop keeps running, just without executing the remaining lines for the current item.

```
# Lecture example: Skip 'Banana'
```

```
fruits = ['Apple', 'Mango', 'Banana', 'Papaya', 'Guava']
```

Same list as before

```
for fruit in fruits:
```

```
    if fruit == 'Banana':
```

```
        continue
```

```
    print(fruit)
```

Check if current item should be skipped

Jump directly to next iteration — 'Banana' is never printed

Runs for all fruits EXCEPT Banana — prints 4 names

```
# Practical: Print only even numbers from 1 to 10
```

```
for i in range(1, 11):
```

```
    if i % 2 != 0:
```

```
        continue
```

```
    print(i)
```

If i is odd (remainder is not 0)...

...skip it and go to the next number

Only even numbers reach this line: 2, 4, 6, 8, 10

TRY THIS LIVE

Create a list: scores = [88, -5, 72, 101, 45, -1, 90]. Some scores are invalid (negative or over 100). Write a for loop that uses continue to skip any invalid score and prints only the valid ones. After the loop, print the count of valid scores. Hint: you need a counter variable initialised before the loop.

04 pass — The Intentional Placeholder

Python requires every code block to contain at least one statement. If you write an if, for, while, or function definition with an empty body, Python throws a SyntaxError. pass is a legal do-nothing statement that satisfies this requirement, letting you write the structure of your code now and fill in the logic later.

CONCEPT — pass

Imagine you are designing a form for a college admission system. You draw all the boxes and labels first — Name, DOB, Course, Photo — even before deciding exactly what validation each field needs. pass does the same thing in code: it holds the place so your program's structure is complete and parseable, without requiring you to write all the logic immediately.

```
for i in range(5):
```

```
    if i == 3:
```

```
        pass
```

```
    else:
```

Loop structure defined

This branch needs special logic — not written yet

Placeholder — Python accepts this, no error, nothing happens at i=3

```
print(i)
```

Prints 0, 1, 2, 4 — i=3 is silently skipped by pass

```
# Common use: empty function during early development
```

```
def calculate_gst():
```

```
    pass
```

Function shell defined — logic to be written later

Without pass, Python crashes with IndentationError

continue — skip and proceed

The loop continues running after continue. The remaining lines for only the current iteration are skipped. All subsequent items are still processed normally.

Result: loop finishes, some iterations partially skipped.

break — stop entirely

The loop terminates the moment break runs. No further items are visited at all. Execution moves to the first line after the loop block.

Result: loop ends early, remaining items untouched.

05 Combining All Three — A Practical Example

In a real program you will often use break and continue together in the same loop. The following example processes a list of student exam results, skipping invalid entries and stopping when the first distinction (75% or above) is found.

```
results = [55, -1, 63, 48, 102, 78, 91, 40]
```

Mix of valid scores, invalid scores (-1, 102), and distinctions (78, 91)

```
for score in results:
```

```
    if score < 0 or score > 100:
```

Invalid data check

```
    print(f'Invalid score {score} — skipping.')
```

```
        continue
```

Skip this score, process the next one

```
    if score >= 75:
```

Check for distinction

```
    print(f'First distinction found: {score}%')
```

```
        break
```

Found it — no need to check the rest

```
    print(f'Score {score}% — valid but not distinction')
```

Valid scores below distinction threshold

CASE STUDY — Flipkart Search & Filter Engine

Context — Flipkart, headquartered in Bengaluru, is India's largest domestic e-commerce platform with over 45 crore registered users and millions of product listings across hundreds of categories.

Challenge — When a user applies price filters, brand filters, or rating filters, Flipkart must scan millions of product records efficiently. Showing every product and then hiding bad matches in the UI is slow and wasteful — filtering must happen at the data level.

What they did — The product search pipeline iterates through eligible product records. For each product: if it

fails a mandatory filter (out of stock, banned seller, missing required field), a `continue` skips it entirely. If a hard limit is reached — for example, 48 results already collected for this page — a `break` stops the iteration without scanning millions of remaining records unnecessarily. `pass`-equivalents hold filter slots open for features still being developed.

Result — Flipkart's search returns filtered results in under 300ms for most queries, scanning and rejecting millions of records without sending any unnecessary data to the user's screen.

Takeaway — `continue` and `break` are not just loop tricks — they are performance tools. Every large-scale search and recommendation system depends on them to avoid doing unnecessary work.

Source: Flipkart Engineering Blog, tech.flipkart.com; public architecture talks at conferences (2022–23)

“break says 'I am done.' continue says 'Not this one.' pass says 'I will deal with this later.' Three words — three of the most useful decisions a loop can make.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — Week 2, Lecture 4: Loop Control Statements (Prof. Madhavan Mukund, IIT Madras)

Link — <https://nptel.ac.in/courses/106106145>

WHERE TO GO NEXT

You can now control exactly what happens inside any loop — stopping early, skipping items, and holding placeholders. These three statements appear in virtually every non-trivial Python program.

Explore deeper — Read 'break and continue Statements' at docs.python.org/3/tutorial/controlflow.html#break-and-continue-statements-and-else-clauses-on-loops — pay special attention to the `loop else` clause, a Python feature that surprises most beginners

Try on your own — Build a simple login system: store a correct password as a variable. Use `while True` with a counter. Give the user 3 attempts. Use `break` on success, increment on failure, and after 3 failures print 'Account locked' and break out of the loop.

Connect the dots — Session 8 introduces strings in depth — indexing, slicing, and methods. Loops are used constantly with strings, so the skills from the last three sessions all combine immediately

Going further — Search 'Python for-else and while-else' to learn about the unusual but useful `else` clause that loops can have — it only runs if the loop completed **WITHOUT** hitting a `break`

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor for a beginner student in India. Explain in simple Hinglish the difference between break, continue, and pass using three different real-life situations: a security guard stopping everyone at an event when a VIP arrives (break), a delivery agent skipping a cancelled order and moving to the next (continue), and an architect leaving a room blank on a blueprint to fill in later (pass). Keep it under 200 words.

SESSION

08

60 MIN · FUNDAMENTALS

Strings

◆ Learning Objectives

- Access individual characters in a string using positive and negative index positions
- Extract substrings using slicing with start, stop, and step values
- Reverse a string using slice notation
- Apply the most important string methods: upper, lower, strip, replace, split, find, and count
- Combine indexing, slicing, and methods to solve practical string problems

01 Strings as Sequences — Indexing

A string is not just text — it is an ordered sequence of characters, and Python gives every character a numbered position called an index. This means you can reach into any string and pull out any single character, or any range of characters, with precision.

CONCEPT — Index Positions

Think of a string like the reserved seats in a GSRTC bus. Every seat has a number starting from 0 at the front. Seat 0 is the first, seat 1 is the second, and so on. Python also counts backwards from the rear: seat -1 is the last, -2 is second-to-last. You can ask for any specific seat directly without going through all the others first.

<code>city = 'Ahmedabad'</code>	9 characters: A=0, h=1, m=2, e=3, d=4, a=5, b=6, a=7, d=8
<code>print(city[0])</code>	Output: A — first character (index 0)
<code>print(city[4])</code>	Output: d — fifth character (index 4)
<code>print(city[-1])</code>	Output: d — last character (negative index -1)
<code>print(city[-3])</code>	Output: b — third from the end
<code>print(len(city))</code>	Output: 9 — total number of characters

WARNING — IndexError: string index out of range

If you try to access an index that does not exist — for example `city[20]` in a 9-character string — Python throws an `IndexError` and your program crashes. Always check `len(string) - 1` for the last valid positive index. Negative indices go from -1 down to -`len(string)`. Anything outside that range is invalid.

02 Slicing — Extracting Substrings

Slicing extracts a portion of a string — a substring — using the notation `string[start:stop:step]`. Start is included, stop is excluded. If you leave start or stop blank, Python defaults to the beginning or end of the string respectively.

```
city = 'Ahmedabad'      Index: 0123456789 (length 9)
print(city[0:5])        Output: Ahmed   — characters at index 0,1,2,3,4 (stop 5 excluded)
print(city[5:])          Output: abad    — from index 5 to the end
print(city[:5])          Output: Ahmed   — from start up to (not including) index 5
print(city[0:9:2])       Output: Amdbd    — every 2nd character (step=2)
print(city[::-1])        Output: dadbemhA — reverses the entire string (step=-1)
```

CONCEPT — Reversing a String with `[::-1]`

The slice `[::-1]` is Python's idiomatic way to reverse any sequence. Start and stop are both blank (meaning 'full string'), and step is -1 (meaning 'step backwards one character at a time'). It looks unusual at first, but every experienced Python developer uses this pattern — memorise it.

```
# Real use: masking Aadhaar number
aadhaar = '987654321098'
masked = 'XXXX-XXXX-' + aadhaar[-4:]
print(masked)

# Extracting domain from email
email = 'riya@tops-int.com'
at_position = email.find('@')
domain = email[at_position + 1:]
print(domain)
```

Only show last 4 digits — standard fintech practice
12-digit Aadhaar as a string (never store as int)
Slice last 4 characters and prefix with Xs
Output: XXXX-XXXX-1098

Find where @ appears (covered in Section 3)
Everything after the @ sign
Output: tops-int.com

TRY THIS LIVE

Store your full name as a string. Using only slicing: (1) print your first name, (2) print your last name, (3) print your initials (first character of each part), and (4) print your full name reversed. No string methods allowed for this exercise — slicing only.

03 String Methods

Python strings come with built-in methods — functions attached to the string itself, called using dot notation: `string.method()`. Strings are immutable in Python, which means every method returns a new string rather than changing the original. The original variable is never modified.

PRO TIP — Strings Are Immutable

`city = 'ahmedabad'` followed by `city.upper()` does NOT change `city`. It returns a new uppercase version. To keep the result, you must assign it: `city = city.upper()`. This trips up beginners constantly — always assign the result if you need it.

Method	What It Does	Example	Output
<code>upper()</code>	All characters to uppercase	<code>'surat'.upper()</code>	<code>'SURAT'</code>
<code>lower()</code>	All characters to lowercase	<code>'VADODARA'.lower()</code>	<code>'vadodara'</code>
<code>title()</code>	First letter of each word capitalised	<code>'hello world'.title()</code>	<code>'Hello World'</code>
<code>strip()</code>	Removes leading and trailing whitespace	<code>' riya '.strip()</code>	<code>'riya'</code>
<code>replace(a, b)</code>	Replaces every occurrence of a with b	<code>'hello'.replace('l', 'r')</code>	<code>'herro'</code>
<code>split(sep)</code>	Splits string into a list at separator	<code>'a,b,c'.split(',')</code>	<code>['a', 'b', 'c']</code>
<code>find(sub)</code>	Returns index of first match; -1 if absent	<code>'Ahmedabad'.find('bad')</code>	<code>6</code>
<code>count(sub)</code>	Counts how many times sub appears	<code>'banana'.count('a')</code>	<code>3</code>
<code>startswith(sub)</code>	True if string starts with sub	<code>'TOPS'.startswith('TO')</code>	<code>True</code>
<code>endswith(sub)</code>	True if string ends with sub	<code>'tops-int.com'.endswith('.com')</code>	<code>True</code>

```
name = ' Riya Shah '
clean_name = name.strip().title()
print(clean_name)
```

Name with accidental spaces (common from user input)
Chain methods: strip spaces first, then capitalise
Output: Riya Shah — clean and properly formatted

```
email = 'RIYA@GMAIL.COM'
print(email.lower())
```

User typed in all caps (common mistake in forms)
Output: `riya@gmail.com` — normalised for storage and comparison

```
sentence = 'Python is easy and Python is fun'
print(sentence.count('Python'))
print(sentence.replace('Python', 'Coding'))
```

Output: 2 — Python appears twice
Output: Coding is easy and Coding is fun

04 Looping Through Strings

Because a string is a sequence, you can loop through it with a for loop exactly as you would a list. Each iteration gives you one character. This lets you analyse, transform, or count characters with full control.

```
# Count vowels in a word
word = input('Enter a word: ')
vowels = 'aeiouAEIOU'
vowel_count = 0
for char in word:
    if char in vowels:
        vowel_count += 1
print(f'Vowels in "{word}": {vowel_count}')
```

Classic string + loop example

All vowels (upper and lower) in one reference string

Initialise counter before the loop

Visit each character one by one

'in' checks if char exists anywhere in the vowels string

Increment only for vowels

05 Practical String Exercises

◆ Reverse a String

```
text = input('Enter any text: ')
reversed_text = text[::-1]
print(f'Reversed: {reversed_text}')
```

Collect input

Reverse using the slice shortcut

```
# Check if it is a palindrome
if text.lower() == reversed_text.lower():
    print('This is a palindrome!')
else:
    print('Not a palindrome.')
```

A palindrome reads the same forwards and backwards

Normalise case before comparing

e.g. 'racecar', 'madam', 'Nitin'

◆ Substring Extraction — Parsing a UPI ID

```
upi_id = 'riya.shah@okaxis'
at_pos = upi_id.find('@')
username = upi_id[:at_pos]
bank = upi_id[at_pos + 1:]
print(f'Username : {username}')
print(f'Bank VPA : {bank}')
```

Sample UPI ID — format is username@bank

Find the @ sign position

Everything before @ = 'riya.shah'

Everything after @ = 'okaxis'

Output: riya.shah

Output: okaxis

TRY THIS LIVE

Build a 'Username Validator': ask the user to enter a desired username. Using string methods and conditions, check that: (1) the username is at least 5 characters long (`len`), (2) it contains no spaces (use `' '` not in `username`), and (3) it does not start with a number (check `username[0]` not in `'0123456789'`). Print a specific failure message for each rule that is broken, or 'Username accepted!' if all three pass.

CASE STUDY — NPCI UPI ID Validation

Context — The National Payments Corporation of India (NPCI) operates the UPI network, which processed over 11,761 crore transactions worth Rs 182 lakh crore in FY 2023–24. Every payment on PhonePe, Google Pay, and Paytm runs through NPCI's validation layer.

Challenge — A UPI ID (VPA) must conform to a strict format: a username part, followed by exactly one '@' symbol, followed by a bank identifier. Millions of user-entered UPI IDs must be validated before any transaction is initiated — a single malformed ID reaching the payment network causes a failed transaction and degrades trust.

What they did — The validation layer applies a sequence of string operations to every UPI ID entered: `find()` to locate the '@' and confirm it exists exactly once, slicing to separate the username and bank handle, `strip()` to remove accidental whitespace, `lower()` to normalise case for comparison, and `startswith()/endswith()` to check handle formats against a list of registered bank identifiers.

Result — UPI rejections due to malformed IDs are under 0.5% of total attempts — an extraordinarily low error rate across billions of transactions — because bad inputs are caught by string validation before they ever reach the payment network.

Takeaway — Every form field you build that accepts text — email, phone, name, UPI ID — needs string methods to clean and validate it. This is the most frequently used set of Python skills in any web or data project.

Source: NPCI Annual Report 2023–24, npci.org.in; RBI Digital Payment Statistics 2024

“Almost all data that enters a program comes in as a string. Knowing how to slice it, clean it, and search it is the first skill every real project demands.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — Week 3, Lecture 1: Strings in Python (Prof. Madhavan Mukund, IIT Madras)

Link — <https://nptel.ac.in/courses/106106145>

WHERE TO GO NEXT

Strings are the most common data type you will work with in any real project. The indexing and slicing logic you learned here applies directly to lists, tuples, and any other sequence type in Python.

Explore deeper — Read the full string methods reference at docs.python.org/3/library/stdtypes.html#string-methods — there are over 40 methods. Scan the list once so you know what exists before you need it.

Try on your own — Build a 'Name Card Generator': ask for first name, last name, and job title. Output a formatted card using f-strings, methods, and slicing — showing initials (`first[0].upper() + last[0].upper()`), the full name in title case, and the title in uppercase.

Connect the dots — Session 9 introduces lists — Python's most versatile collection type. The indexing and slicing you just learned for strings works exactly the same way on lists, so the transition will feel natural.

Going further — Search 'Python regular expressions re module' — the next level of string pattern matching used in real validation systems for emails, phone numbers, and PAN numbers

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor for a beginner student in India. Explain in simple Hinglish how string slicing works using the analogy of cutting a piece of fabric: you specify where to start cutting, where to stop, and how big each step is. Use 'Ahmedabad' as the example string and show what `city[0:5]`, `city[-4:]`, and `city[::-1]` produce and why. Keep it under 200 words.

SESSION

09

60 MIN · FUNDAMENTALS

Lists

◆ Learning Objectives

- Create a list and access items using positive and negative index positions
- Add items to a list using `append()` and `insert()`
- Remove items from a list using `remove()` and `pop()`
- Sort a list in place with `sort()` and explain how it differs from `sorted()`
- Loop through a list and combine list operations with conditional logic
- Explain the difference between a list and a string as sequence types

01 What is a List?

A list is an ordered, changeable collection that can hold any number of values — including mixed types. It is Python's most versatile and widely used data structure. You will use lists in virtually every real project you build, from storing form submissions to processing rows of data.

CONCEPT — List as an Ordered Collection

Think of a list like an attendance register in a TOPS classroom. Each row has a position (index 0, 1, 2...), names can be added when a new student joins, removed when someone leaves, and the whole register can be sorted alphabetically. Unlike a string — which holds only characters — a list can hold names, numbers, booleans, and even other lists all at once.

Creating lists

```
subjects = ['Python', 'SQL', 'Django', 'HTML', 'CSS']
```

List of strings

```
marks = [88, 74, 91, 65, 83]
```

List of integers

```
mixed = ['Riya', 20, 8.75, True]
```

Mixed types — valid in Python

```
empty = []
```

An empty list — you add items later

Accessing items

```
print(subjects[0])
```

Output: Python — first item (index 0)

```
print(subjects[-1])
```

Output: CSS — last item (negative index)

```
print(subjects[1:4])
```

Output: ['SQL', 'Django', 'HTML'] — slicing works exactly like strings

```
print(len(subjects))
```

Output: 5 — total number of items

PRO TIP — Membership Test with 'in'

To check whether an item exists in a list without looping through it manually, use the `in` operator: `'Django' in subjects` returns `True`. This is faster to write and more readable than a `for` loop with a flag variable. You will use this pattern constantly in real code.

02 Adding Items — `append()` and `insert()`

Lists are mutable — you can change them after creation. `append()` adds one item to the end of the list. `insert()` places an item at any specific position, pushing existing items to the right.

```
squad = ['Shubman Gill', 'David Miller', 'Rashid Khan']
```

Gujarat Titans starting squad

```
squad.append('Mohammed Shami')
print(squad)
```

Add Shami to the END of the list
['Shubman Gill', 'David Miller', 'Rashid Khan',
'Mohammed Shami']

```
squad.insert(0, 'Hardik Pandya')
print(squad)
```

Insert Hardik at index 0 (becomes the first item)
['Hardik Pandya', 'Shubman Gill', 'David Miller', 'Rashid
Khan', 'Mohammed Shami']

```
squad.insert(2, 'Vijay Shankar')
print(len(squad))
```

Insert at index 2 — shifts everything after it right by one
Output: 6 — list grew by one

CONCEPT — `append()` vs `insert()`

`append()` is $O(1)$ — it adds to the end in constant time regardless of how big the list is. `insert(0, x)` is $O(n)$ — it has to shift every existing item one position to the right. For large lists, always prefer `append()` over `insert(0, x)`. If you find yourself inserting at the front frequently, consider a different data structure (deque from the `collections` module).

03 Removing Items — `remove()` and `pop()`

Python gives you two ways to remove items from a list. `remove()` takes a value and deletes the first matching occurrence. `pop()` takes an index (or removes the last item if no index is given) and returns the removed value — useful when you need to know what was deleted.

Method	Removes By	Returns	Error If Not Found
<code>remove(value)</code>	Value — first match	Nothing (None)	<code>ValueError</code>

Method	Removes By	Returns	Error If Not Found
pop()	Last item (no index)	The removed item	IndexError if list is empty
pop(index)	Item at given index	The removed item	IndexError if index invalid
del list[i]	Item at index i	Nothing	IndexError if index invalid
clear()	All items — list becomes []	Nothing	Never

```
squad = ['Hardik', 'Shubman', 'Miller', 'Rashid', 'Shami']
```

```
squad.remove('Miller')
```

Finds 'Miller' by value and removes it — error if not in list

```
print(squad)
```

['Hardik', 'Shubman', 'Rashid', 'Shami']

```
released = squad.pop()
```

Removes and returns the LAST item

```
print(released)
```

Output: Shami

```
print(squad)
```

['Hardik', 'Shubman', 'Rashid']

```
transferred = squad.pop(1)
```

Removes and returns item at index 1

```
print(transferred)
```

Output: Shubman

WARNING — remove() Raises ValueError for Missing Items

If you call `squad.remove('Kohli')` and 'Kohli' is not in the list, Python raises a `ValueError` and crashes. Always check first: if 'Kohli' in `squad`: `squad.remove('Kohli')`. Alternatively, use a try-except block — but check with 'in' for readability at this stage.

04 Sorting Lists

Python provides two ways to sort a list: the `sort()` method modifies the list in place and returns `None`. The `sorted()` function returns a new sorted list and leaves the original untouched. Choosing the wrong one is a very common bug.

sort() — modifies in place

```
marks = [55, 88, 42, 91]
marks.sort()
print(marks) # [42, 55, 88, 91]
```

The original list IS the sorted list now. Returns `None` — so `result = marks.sort()` gives you `None`, not a sorted list.

sorted() — returns a new list

```
marks = [55, 88, 42, 91]
result = sorted(marks)
print(result) # [42, 55, 88, 91]
print(marks) # [55, 88, 42, 91] — unchanged!
```

Use `sorted()` when you need the original order preserved.

```
cities = ['Surat', 'Ahmedabad', 'Rajkot', 'Vadodara', 'Bhavnagar']
```

Gujarat cities in no particular order

```
cities.sort()
```

Sort alphabetically in place (A to Z)

```
print(cities)
```

['Ahmedabad', 'Bhavnagar', 'Rajkot', 'Surat', 'Vadodara']

```
cities.sort(reverse=True)
```

Sort in reverse (Z to A)

```
print(cities)
```

['Vadodara', 'Surat', 'Rajkot', 'Bhavnagar', 'Ahmedabad']

```
marks = [72, 88, 45, 91, 60]
```

```
top_marks = sorted(marks, reverse=True)
```

Sorted copy descending — marks is unchanged

```
print(top_marks)
```

[91, 88, 72, 60, 45]

```
print(marks)
```

[72, 88, 45, 91, 60] — original untouched

05 Looping Through Lists & Practical Patterns

```
marks = [82, 45, 91, 33, 67, 78, 29, 88]
```

Student marks list

```
total = 0
```

Accumulator for total

```
for score in marks:
```

Loop through every score

```
    total += score
```

Add each score to total

```
average = total / len(marks)
```

Divide total by number of students

```
highest = max(marks)
```

Built-in max() — no manual loop needed

```
lowest = min(marks)
```

Built-in min()

```
print(f'Average: {average:.1f}, High: {highest}, Low: {lowest}')
```

TRY THIS LIVE

Create a list called wishlist with five product names you would buy online. Then: (1) add a sixth product at the end with append(), (2) insert another at index 0, (3) remove one by name with remove(), (4) pop the last item and print what was removed, (5) sort the list alphabetically and print it. Do each step one at a time and print the list after each operation to watch it change.

CASE STUDY — Gujarat Titans IPL Auction & Squad Management

Context — Gujarat Titans (GT), based in Ahmedabad, won the IPL in their debut season (2022) and returned to the final in 2023. Their team management builds and adjusts a squad of up to 25 players through multiple auction rounds, retention decisions, and mid-season replacements.

Challenge — The franchise must maintain an ordered squad list at all times — tracking retained players, auction purchases, injury replacements, and releases — while respecting the cap on foreign players per

match (max 4) and the overall salary cap. Adding, removing, and re-sorting the squad by role, price, or availability happens constantly.

What they did — Their analytics team maintains structured player lists segmented by role (batters, bowlers, all-rounders, wicket-keepers). When a player is purchased at auction, `append()` adds them. When released, `remove()` drops them by name. When injuries occur, `insert()` places a replacement at the correct position. Before each match, the squad list is sorted by performance rating to assist selection.

Result — GT fielded consistently optimised playing XI selections, with their data-driven squad management cited as a key factor in back-to-back finals appearances — a result no other IPL franchise achieved in their first two seasons.

Takeaway — A cricket squad, a product catalogue, a student waitlist — they are all ordered, changeable collections of items. The list operations you learned today are the same operations that manage them in production systems.

Source: Gujarat Titans official communications; ESPNcricinfo squad data 2022–23; BCCI IPL auction records

“The list is Python's Swiss Army knife. You will reach for it first in almost every problem — and most of the time, it will be exactly what you need.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — Week 3, Lecture 2: Lists in Python (Prof. Madhavan Mukund, IIT Madras)

Link — <https://nptel.ac.in/courses/106106145>

WHERE TO GO NEXT

Lists are changeable. The next session introduces tuples — sequences that are intentionally locked after creation, and why that difference matters.

Explore deeper — Read the full list methods reference at docs.python.org/3/tutorial/datastructures.html#more-on-lists — especially the section on using lists as stacks and queues

Try on your own — Build a 'Student Waitlist Manager': start with an empty list. Let the user type a name to add to the end, 'remove' followed by a name to remove someone, 'first' to pop the first person off the waitlist, or 'show' to print the current list. Use a while True loop and break to exit on 'quit'.

Connect the dots — Session 10 introduces tuples — a close cousin of the list that cannot be changed after creation. You will learn when to choose each and why immutability is sometimes exactly what you want.

Going further — Search 'Python list comprehension' to see a powerful one-line way to create and filter lists — covered in depth in Module 2, but worth a preview

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor for a beginner student in India. Explain in simple Hinglish the difference between `append()` and `insert()` for lists, and between `sort()` and `sorted()`. Use the example of managing a cricket team squad — adding players to the end after an auction (`append`), inserting a captain at position 0 (`insert`), and sorting by jersey number (`sort` vs `sorted`). Keep it under 200 words.

SESSION

10

60 MIN · FUNDAMENTALS

Tuples

◆ Learning Objectives

- Create a tuple and explain how it differs from a list
- Access tuple values using index positions and negative indexing
- Demonstrate immutability and explain why it is a feature, not a limitation
- Extract portions of a tuple using slicing
- Combine tuples using concatenation
- Apply tuple unpacking to assign multiple variables in one line
- Identify real scenarios where a tuple is the correct choice over a list

01 What is a Tuple?

A tuple is an ordered, immutable sequence — like a list, but one that cannot be changed after creation. You cannot add, remove, or replace items. This is not a limitation — it is a deliberate design choice that signals intent: the data in this container is fixed and should stay that way.

CONCEPT — Immutability as a Signal

Think of a train ticket from Ahmedabad to Mumbai. The departure station, arrival station, date, and seat number are printed and fixed — you cannot edit them after booking. The ticket is immutable by design because these facts should never change mid-journey. A tuple works the same way: use it when the data is a fixed, complete record that must stay exactly as created.

List [] — mutable

Use when the collection needs to grow, shrink, or change over time.

Examples: enrolled students, items in a cart, task queue, search results.

You expect to call `append()`, `remove()`, `sort()` on it.

Tuple () — immutable

Use when the data is a fixed record that must not change.

Examples: GPS coordinates, RGB colour values, a database row, (latitude, longitude) of a city.

If anyone changes it, something is wrong.

02 Creating Tuples & Accessing Values

Tuples are created with parentheses () instead of square brackets []. Accessing items uses exactly the same index and negative-index syntax as lists and strings — the sequence indexing rules are consistent across all Python sequence types.

```
# Creating tuples — Gujarat city coordinates
ahmedabad = (23.0225, 72.5714)
surat      = (21.1702, 72.8311)
rajkot     = (22.3039, 70.8022)

print(ahmedabad[0])
print(ahmedabad[1])
print(ahmedabad[-1])
print(len(ahmedabad))

# A richer record — train timetable entry
train = ('12901', 'Gujarat Mail', 'Ahmedabad', 'Mumbai', '22:10', '07:25')
      Train no, name, from, to, depart, arrive
print(train[1])
print(train[-2])
```

Latitude, Longitude — fixed forever

Output: 23.0225 — latitude (index 0)

Output: 72.5714 — longitude (index 1)

Output: 72.5714 — last item (negative index)

Output: 2 — two elements

Output: Gujarat Mail

Output: 22:10 — departure time

PRO TIP — Single-Item Tuple Needs a Trailing Comma

This is the most common tuple mistake: `single_item = ('hello')` is just a string in parentheses — Python ignores the brackets. To create a tuple with one item, add a trailing comma: `single_item = ('hello',)`. The comma is what makes it a tuple, not the parentheses. Run `type(('hello'))` and `type(('hello',))` to see the difference yourself.

03 Immutability — Why It Matters

Attempting to modify a tuple raises a `TypeError` immediately. This is Python enforcing correctness: if you accidentally try to change data that should be fixed — GPS coordinates, a booked seat, a tax rate — the program stops and tells you rather than silently corrupting the data.

```
ahmedabad = (23.0225, 72.5714)
ahmedabad[0] = 25.0

# Tuples cannot grow or shrink
ahmedabad.append(500)
ahmedabad.remove(72.5714)
```

`TypeError: 'tuple' object does not support item assignment`

`AttributeError: 'tuple' object has no attribute 'append'`

`AttributeError: 'tuple' object has no attribute 'remove'`

CONCEPT — Tuples as Dictionary Keys

One major practical advantage of immutability: tuples can be used as dictionary keys, but lists cannot. Because a tuple can never change, Python can calculate a permanent hash for it. This is used constantly in mapping systems — for example, a route planner stores distances using `(city_a, city_b)` as the key:

distances[(23.02, 72.57), (21.17, 72.83)] = 265. Lists are unhashable and cannot do this.

04 Tuple Slicing

Slicing a tuple works identically to slicing a string or a list — the same start:stop:step syntax applies. Slicing always returns a new tuple, never a list. This consistency across sequence types is intentional — once you know slicing, you know it for everything.

```
train = ('12901', 'Gujarat Mail', 'Ahmedabad', 'Mumbai', '22:10', '07:25')
Full train record

route = train[2:4]
print(route)

timings = train[4:]
print(timings)

identity = train[:2]
print(identity)

reversed_train = train[::-1]
print(reversed_train)
```

Extract origin and destination only
Output: ('Ahmedabad', 'Mumbai')

Extract departure and arrival times
Output: ('22:10', '07:25')

Train number and name only
Output: ('12901', 'Gujarat Mail')

Reverse the whole tuple
Output: ('07:25', '22:10', 'Mumbai', 'Ahmedabad', 'Gujarat Mail', '12901')

05 Tuple Concatenation & Unpacking

Tuples cannot be changed in place, but you can create a new tuple by combining two existing ones with the + operator. You can also unpack a tuple — assign each of its values to a separate variable in a single line. Unpacking is one of the most used Python patterns in real codebases.

◆ Concatenation

```
gujarat_cities = ('Ahmedabad', 'Surat', 'Rajkot')
First tuple — unchanged
more_cities = ('Vadodara', 'Bhavnagar', 'Gandhinagar')
Second tuple

all_cities = gujarat_cities + more_cities
print(all_cities)
```

Concatenation — creates a NEW tuple, originals intact
('Ahmedabad', 'Surat', 'Rajkot', 'Vadodara', 'Bhavnagar', 'Gandhinagar')

```
repeated = ('TOPS',) * 3
print(repeated)
```

Repetition — creates ('TOPS', 'TOPS', 'TOPS')

Output: ('TOPS', 'TOPS', 'TOPS')

◆ Tuple Unpacking

```
ahmedabad = (23.0225, 72.5714)
latitude, longitude = ahmedabad
print(latitude)
print(longitude)
```

Two values in one tuple

Unpack — assigns both in one line (must match count)

Output: 23.0225

Output: 72.5714

```
# Swap two variables — Python's elegant trick
```

```
a = 10
b = 20
a, b = b, a
```

Swap in one line — Python packs (b, a) into a tuple then unpacks

Output: 20 10

```
# Unpacking a database-style row
row = ('2024CS047', 'Riya Shah', 8.75, 'Surat')
```

As returned from a database query

```
roll, name, cgpa, city = row
print(f'{name} from {city} — CGPA {cgpa}')
```

Clean, readable unpacking of each field

Output: Riya Shah from Surat — CGPA 8.75

TRY THIS LIVE

Create a tuple called `student_record` with five fields: roll number (str), name (str), percentage (float), city (str), and passed (bool). Use tuple unpacking to assign all five to individual variables in one line. Then print a formatted summary using an f-string. Finally, try to change the percentage to a higher value and observe the `TypeError`.

CASE STUDY — Google Maps & GPS Coordinates as Tuples

Context — Google Maps processes over 1 billion kilometres of directions every day. In India, Google Maps is used by Ola, Rapido, Dunzo, and IRCTC for real-time location tracking and route calculation. The Gujarat road network — including the Ahmedabad-Vadodara Expressway — is fully mapped with coordinate-level precision.

Challenge — Every location on earth is represented as a (latitude, longitude) pair. These coordinates are the foundation of every map lookup, route calculation, and distance measurement. The values must never be accidentally modified — a coordinate that changes mid-route causes the navigation to jump to a wrong location, potentially with serious consequences.

What they did — Location data is stored and passed as tuples throughout the navigation pipeline. The

immutability of a tuple guarantees that (23.0225, 72.5714) for Ahmedabad will never become (25.0, 72.5714) due to a bug in one part of the system. When routes need to be built, tuples of waypoints are concatenated: `route = origin_coords + waypoints + destination_coords`.

Result — Google Maps' location accuracy in India is within 5 metres in most urban areas. The architectural decision to treat coordinates as immutable records — not editable collections — is a key reason the system remains consistent across billions of lookups.

Takeaway — When the correctness of data matters more than the ability to change it, use a tuple. Immutability is not a restriction — it is a guarantee that protects the entire system.

Source: Google Maps Platform documentation; Google India blog; NITI Aayog Geospatial report 2022

“A list says ‘these items may change.’ A tuple says ‘these items are facts.’ Choose based on what the data is — not just what is convenient to write.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — Week 3, Lecture 3: Tuples in Python (Prof. Madhavan Mukund, IIT Madras)

Link — <https://nptel.ac.in/courses/106106145>

WHERE TO GO NEXT

You now have both mutable (list) and immutable (tuple) sequence types. Session 11 introduces dictionaries — a completely different kind of collection where items are accessed by a meaningful name, not a number.

Explore deeper — Read docs.python.org/3/tutorial/datastructures.html#tuples-and-sequences — pay attention to the section explaining when tuples are used by Python itself internally

Try on your own — Create a tuple containing the details of your favourite movie or cricket match: title, year, director (or winner), and rating. Unpack it into four variables and print a formatted summary. Then try to update the rating — observe and note the error message you get.

Connect the dots — Session 11 covers dictionaries — a data structure where values are accessed by key names like `student['name']` instead of position numbers like `student[0]`. Tuples are frequently used as dictionary keys, so the two concepts connect directly.

Going further — Search 'Python namedtuple collections' — a way to give each position in a tuple a human-readable name, giving you the best of tuples and the readability of dictionaries

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor for a beginner student in India. Explain in simple Hinglish why tuples are immutable and when you should choose a tuple over a list. Use two relatable Indian examples: GPS coordinates of Ahmedabad (should never change — tuple) versus a student's subject list (can change each semester — list). Show the code for both. Keep it under 200 words.

SESSION

11

60 MIN · FUNDAMENTALS

Dictionaries

◆ Learning Objectives

- Explain what a dictionary is and why key-based access is more readable than index-based access
- Create a dictionary and access values using keys
- Add new key-value pairs and update existing values
- Use `get()` to safely retrieve values without risking a `KeyError`
- Remove entries using `del` and `pop()`
- Iterate through a dictionary using `keys()`, `values()`, and `items()`
- Build a multi-record structure using a list of dictionaries

01 What is a Dictionary?

A dictionary stores data as key-value pairs. Instead of accessing a value by its position number (like index 2), you access it by a meaningful name — a key. This makes dictionaries the natural structure for representing real-world objects: a student profile, a product listing, a user account, a configuration file.

CONCEPT — Key-Value Pairs

Think of a dictionary exactly like an Aadhaar card. The card has labelled fields — 'Name', 'DOB', 'Gender', 'Address' — and each field has a value. You do not look up the name by asking for 'field 0'; you ask for the 'Name' field directly. That label is the key. Python dictionaries work the same way: fast, direct lookup by a meaningful name, not a positional guess.

```
student = {
    'name' : 'Riya Shah',
    'roll' : '2024CS047',
    'cgpa' : 8.75,
    'city' : 'Surat',
    'enrolled': True
}
```

Opening curly brace — starts the dictionary

key: value — key is always a string here (keys must be immutable)

No comma after the last pair

Closing curly brace — ends the dictionary

```
print(student['name'])
print(student['cgpa'])
print(type(student))
```

Output: Riya Shah — access by key name

Output: 8.75

Output: <class 'dict'>

WARNING — KeyError When Accessing a Missing Key

`student['age']` raises a `KeyError` and crashes your program if 'age' is not a key in the dictionary. Unlike a list

where an out-of-range index is unusual, accessing a missing dictionary key is a very common situation — for example, an optional field the user chose not to fill. Always use `get()` for keys that might not exist (covered in Section 3).

02 Adding & Updating Entries

Dictionaries are mutable. Assigning to a new key adds it. Assigning to an existing key overwrites its value. There is no separate 'add' and 'update' method for basic operations — the same assignment syntax handles both, and Python decides which to do based on whether the key already exists.

```
student = {'name': 'Riya Shah', 'cgpa': 8.75, 'city': 'Surat'}
```

Starting dictionary

```
# Adding a new key
```

```
student['phone'] = '9876543210'
```

```
student['course'] = 'Python'
```

```
print(student)
```

'phone' does not exist — Python creates it

Another new key added

Now has 5 keys

```
# Updating an existing key
```

```
student['cgpa'] = 9.1
```

```
student['city'] = 'Ahmedabad'
```

```
print(student['cgpa'])
```

'cgpa' already exists — Python replaces 8.75 with 9.1

Riya moved cities — update reflects this

Output: 9.1 — old value is gone

03 Safe Access with `get()`

`get()` is the safe alternative to bracket access. It returns the value if the key exists, and returns `None` (or a default you specify) if it does not — without raising an error. This is the professional way to handle optional or user-provided data.

```
student = {'name': 'Riya Shah', 'cgpa': 8.75}
```

'age' and 'phone' not present

```
# Bracket access — CRASHES if key missing
```

```
print(student['phone'])
```

`KeyError: 'phone'` — program stops here

```
# get() — SAFE access
```

```
print(student.get('phone'))
```

Output: `None` — key missing, no crash

```
print(student.get('phone', 'Not provided'))
```

Output: `Not provided` — custom default when key is absent

```
print(student.get('name', 'Unknown'))
```

Output: `Riya Shah` — key exists, default ignored

```
# Check if a key exists before using it
```

```
if 'cgpa' in student:
    print(f'CGPA: {student["cgpa"]}')
```

'in' checks keys — returns True or False
Safe to use bracket access after confirming key exists

PRO TIP — Use get() for All Optional Fields

Any time a key might not exist — a form field the user left blank, an optional API response field, a config option with a fallback — use get() with a default. It is shorter than an if 'key' in dict check and reads more clearly. Real-world API responses and database queries are full of optional fields; get() is the tool that handles them cleanly.

04 Removing Entries

```
student = {'name': 'Riya', 'cgpa': 8.75, 'city': 'Surat', 'temp': 'delete me'}
```

del — remove by key, no return value

```
del student['temp']
print(student)
```

Removes the 'temp' key permanently
{'name': 'Riya', 'cgpa': 8.75, 'city': 'Surat'}

pop() — remove and RETURN the value

```
removed_city = student.pop('city')
print(removed_city)
print(student)
```

Removes 'city' and returns its value
Output: Surat — the removed value
{'name': 'Riya', 'cgpa': 8.75}

pop() with a default — safe even if key missing

```
val = student.pop('phone', 'Not found')
print(val)
```

No KeyError if 'phone' does not exist
Output: Not found

05 Iterating — keys(), values(), items()

A dictionary can be looped through in three ways depending on what you need from each iteration. items() is by far the most commonly used because it gives both the key and the value together on every pass.

Method	What It Returns	Use When You Need
keys()	A view of all keys	Just the field names — e.g. checking which fields are present
values()	A view of all values	Just the data — e.g. summing all marks, checking all cities

Method	What It Returns	Use When You Need
<code>items()</code>	A view of (key, value) tuples	Both key and value together — the most common loop case

```
student = {'name': 'Riya Shah', 'roll': '2024CS047', 'cgpa': 8.75, 'city': 'Surat'}
```

```
# Loop through keys only
```

```
for key in student.keys():
    print(key)
```

Prints: name, roll, cgpa, city

```
# Loop through values only
```

```
for value in student.values():
    print(value)
```

Prints: Riya Shah, 2024CS047, 8.75, Surat

```
# Loop through both — most common pattern
```

```
for key, value in student.items():
    print(f'{key:12} : {value}')
```

Unpacks each (key, value) tuple automatically
:12 pads the key to 12 chars for alignment

◆ A List of Dictionaries — The Most Common Real-World Pattern

In real projects, you almost never work with a single dictionary in isolation. A database table, an API response, or a report is a list of records — meaning a list where every item is a dictionary. This pattern appears in virtually every data, web, and automation project.

```
students = [
    {'name': 'Riya Shah', 'cgpa': 8.75, 'city': 'Surat'},
    {'name': 'Arjun Patel', 'cgpa': 7.20, 'city': 'Ahmedabad'},
    {'name': 'Meera Dave', 'cgpa': 9.10, 'city': 'Rajkot'},
]
```

A list of student records

```
# Print all names
```

```
for student in students:
    print(student['name'])
```

Each iteration: student is one dictionary

Access the 'name' key from each record

```
# Find students from Ahmedabad with cgpa >= 8.0
```

```
for student in students:
    if student['city'] == 'Ahmedabad' and student['cgpa'] >= 8.0:
        print(student['name'], '-', student['cgpa'])
```

TRY THIS LIVE

Build a mini contact book using a list of dictionaries. Each dictionary should have 'name', 'phone', and 'city'.

Add at least four contacts. Then write a loop that searches for all contacts from a specific city (take the city as input from the user) and prints their name and phone number. Use `get()` with a default for the phone field in case it is missing from any record.

CASE STUDY — GSTN Business Registry & GST Lookup

Context — The Goods and Services Tax Network (GSTN) is the technology backbone of India's GST system, managing registrations for over 1.4 crore active taxpayers across every state. Gujarat, as a major trade hub, has over 7 lakh active GSTIN registrations.

Challenge — Every GST-registered business has a unique 15-character GSTIN. When a business files an invoice or a buyer claims an input tax credit, the system must instantly look up the seller's registration details — legal name, state code, registration status, and HSN codes — using only the GSTIN as the lookup key.

What they did — Each taxpayer record is stored as a dictionary: the GSTIN is the key, and the value is a nested dictionary of business details. A lookup call passes the GSTIN string and retrieves the corresponding record in $O(1)$ time — no sequential search through millions of records. `get()` with a default handles invalid or deregistered GSTINs gracefully instead of crashing the invoice processing pipeline.

Result — GSTN processes over 20 crore invoices per month through its e-invoicing portal with sub-second GSTIN lookups, supporting the entire Rs 1.87 lakh crore monthly GST collection for FY 2023–24.

Takeaway — Dictionaries are the universal structure for lookup systems — anywhere you have a unique identifier (GSTIN, Aadhaar, roll number, user ID) that maps to a set of details, a dictionary is the right tool.

Source: GSTN Annual Report 2023–24, gstn.org.in; Ministry of Finance GST collection data 2023–24

“Lists store collections. Tuples store fixed records. Dictionaries store meaning — they are the structure that makes data readable to both the programmer and the code.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — Week 3, Lecture 4: Dictionaries in Python (Prof. Madhavan Mukund, IIT Madras)

Link — <https://nptel.ac.in/courses/106106145>

WHERE TO GO NEXT

You now have all four core data structures: list, tuple, dict, and string. The final session of this module ties everything together with functions — the tool that makes reusable, organised code possible.

Explore deeper — Read the full dictionary documentation at

docs.python.org/3/tutorial/datastructures.html#dictionaries — pay attention to `dict()` constructor syntax and the `update()` method for merging dictionaries

Try on your own — Build a 'Course Catalogue': create a list of five dictionaries, each representing a TOPS course with keys for name, duration (hours), fee (Rs), and seats_available (int). Write a loop that prints only courses where seats_available > 0, and a second loop that prints the average fee across all courses.

Connect the dots — Session 12 — the final session of Module 1 — introduces functions and lambda. Functions let you wrap any of the code from this module into a reusable block you can call by name, making your programs organised and professional.

Going further — Search 'Python defaultdict collections' and 'Python dict comprehension' — two powerful extensions of the basic dictionary that appear constantly in data science and web development code

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor for a beginner student in India. Explain in simple Hinglish what a Python dictionary is and why it is more useful than a list for storing a student's profile. Show a small code example of a student dictionary with name, roll, and city, and demonstrate how to access a value using a key and how to update it. Keep it under 200 words.

SESSION

12

60 MIN · FUNDAMENTALS

Functions & Lambda

◆ Learning Objectives

- Explain what a function is and why reusability is the core reason to write one
- Define a function with `def`, call it, and pass values using positional and keyword arguments
- Set default parameter values so a function works with or without certain inputs
- Return single and multiple values from a function and use them in the caller
- Write a lambda — an anonymous single-expression function — and know when to prefer it
- Apply `map()`, `filter()`, and `reduce()` with lambda to transform and filter collections

01 What is a Function — and Why Write One?

A function is a named, reusable block of code. You define it once and call it by name as many times as you need — with different inputs each time. Without functions, any logic you need in two places has to be copied and pasted, which means two places to update when something changes and two places where bugs can hide.

CONCEPT — The Three Reasons to Write a Function

1. Reusability — write once, call anywhere. A GST calculation written as a function is called the same way for Rs 500 or Rs 50,000. 2. Readability — `calculate_gst(price)` tells the next developer exactly what is happening. A raw formula does not. 3. Maintainability — when the GST rate changes, you update one function. Without it, you hunt through every file for every hardcoded rate. Think of a function like a TOPS trainer: trained once, deployed in every batch, always does the same job correctly.

SOFTWARE MAINTENANCE COST

60–80%

Share of total software cost spent on maintenance — IEEE Software Engineering Body of Knowledge. Functions that are readable and reusable directly reduce this cost.

02 Defining & Calling Functions

A function is defined with the `def` keyword, followed by its name, parentheses for parameters, and a colon. The indented block underneath is the function body — it runs only when the function is called. Defining a function does nothing by itself; you must call it.

DEFINE the function — happens once

```
def greet(name):
```

'name' is a parameter — a placeholder for the real value

```
    message = f'Hello, {name}! Welcome to TOPS.'
```

Indented body — runs only when the function is called

```
        print(message)
```

CALL the function — as many times as needed

```
greet('Riya')
```

Argument 'Riya' is passed in — 'name' becomes 'Riya' inside

```
greet('Arjun')
```

Same function, different input — prints a different greeting

```
greet('Meera')
```

Three outputs, one function definition

WARNING — Parameters vs Arguments

Parameter is the variable name in the function definition: `def greet(name)`. Argument is the actual value passed when calling: `greet('Riya')`. The terms are often used interchangeably in conversation, but understanding the distinction helps when reading error messages — Python will say 'missing 1 required positional argument' when you forgot to pass something.

03 Return Values

A function that only prints is limited — the output disappears after the call. A function that returns a value gives you something you can store, pass to another function, or include in further calculations. The return statement sends a value back to the caller and immediately exits the function.

```
def calculate_gst(price, rate):
    gst_amount = price * rate / 100
    return gst_amount
```

Two parameters — price and tax rate

Calculation happens inside

Sends the result BACK to the caller — function ends here

```
tax = calculate_gst(1000, 18)
print(f'GST on Rs 1000 at 18%: Rs {tax}')
```

Return value is stored in 'tax'

Output: GST on Rs 1000 at 18%: Rs 180.0

Functions can return multiple values — as a tuple

```
def calculate_bill(price, rate=18):
    gst = price * rate / 100
    total = price + gst
    return price, gst, total
```

Returns a tuple of three values

```
base, tax, bill = calculate_bill(500)
```

Unpack — each variable receives one returned value

```
print(f'Base: Rs {base} GST: Rs {tax} Total: Rs {bill}')
```

Output: Base: Rs 500 GST: Rs 90.0 Total: Rs 590.0

PRO TIP — A Function That Does Not Return Still Returns None

If a function has no return statement, Python silently returns None. This is why `result = greet('Riya')` gives you None in result, not the printed string. `print()` is a function too — and it returns None every time. Use `return` when you need the output to travel anywhere beyond the function's own print statement.

04 Types of Arguments — Positional, Keyword & Default

Python gives you three ways to pass values into a function. Positional arguments match by order. Keyword arguments match by name — order does not matter. Default parameter values let a function work even when an argument is not provided.

```
def enrol_student(name, course, city='Ahmedabad'):
    city has a default — it is optional at call time
    print(f'{name} enrolled in {course} ({city})')
```

Positional — order must match the definition

```
enrol_student('Riya', 'Python')           city uses default: Ahmedabad
enrol_student('Arjun', 'Django', 'Surat')  city overridden: Surat
```

Keyword — name each argument explicitly

```
enrol_student(course='SQL', name='Meera')  Order swapped — Python matches by name, not
                                           position
```

```
enrol_student('Dev', city='Rajkot', course='AWS')
```

Mix of positional and keyword — positional must come first

Argument Type	Syntax at Call	Order Matters?	Best Used When
Positional	<code>greet('Riya', 'Surat')</code>	Yes — strictly	Simple functions with 1–3 parameters in an obvious order
Keyword	<code>greet(city='Surat', name='Riya')</code>	No	Many parameters or when skipping optional ones mid-list
Default value	<code>def f(rate=18)</code> — set at definition	N/A	Parameters with a sensible fallback (rate, currency, city)

TRY THIS LIVE

Write a function called `calculate_emi(principal, rate, months=12)` that calculates a simple loan EMI: $EMI = (principal + principal * rate / 100) / months$. Call it three ways: (1) positional only — `calculate_emi(100000, 10)`, (2) all three positional — `calculate_emi(100000, 10, 24)`, and (3) keyword — `calculate_emi(months=36, principal=200000, rate=8)`. Print the EMI formatted to 2 decimal places each time.

05 Lambda Functions

A lambda is a small, anonymous function written in a single expression. It has no name, no `def`, and no return statement — the expression itself is the return value. Lambdas are not a replacement for regular functions; they are a shorthand for simple, throwaway logic used in one place, typically as an argument to another function.

Regular def function

```
def square(x):
    return x * x

result = square(5)
print(result) # 25
```

Use when: the logic is complex, more than one expression, or the function is called from many places.

Lambda

```
square = lambda x: x * x

result = square(5)
print(result) # 25
```

Use when: the function is simple (one expression), used in one place, and often passed directly to `map()`, `filter()`, or `sorted()`.

```
# Lambda with one parameter
double = lambda x: x * 2
print(double(7))

# Lambda with two parameters
add_gst = lambda price, rate: price * rate / 100
print(add_gst(500, 18))

# Lambda with a condition (ternary expression)
grade = lambda marks: 'Pass' if marks >= 45 else 'Fail'
print(grade(72))
print(grade(30))
```

Takes x, returns x * 2
Output: 14

Takes price and rate, returns GST amount
Output: 90.0

One-liner conditional inside a lambda
Output: Pass
Output: Fail

06 map(), filter(), and reduce()

These three built-in functions apply a function across a collection. They are the reason lambdas exist as a concept — they are designed to receive a function as their first argument. Using them with lambda gives you powerful data transformation in one clean line.

◆ map() — Apply a Function to Every Item

```
prices = [100, 250, 500, 1200, 2000]           Base prices before GST

# Using map() – applies the lambda to EVERY item
with_gst = list(map(lambda p: round(p * 1.18, 2), prices))
# Adds 18% GST to each price
print(with_gst)                                [118.0, 295.0, 590.0, 1416.0, 2360.0]

# Equivalent using a for loop – map() is just more concise
with_gst_loop = []
for p in prices:
    with_gst_loop.append(round(p * 1.18, 2))
```

◆ filter() — Keep Only Items That Pass a Test

```
marks = [82, 45, 91, 33, 67, 78, 29, 88]       Class marks

passing = list(filter(lambda m: m >= 45, marks))
# Keeps only marks 45 and above
print(passing)                                  [82, 45, 91, 67, 78, 88]

high_scorers = list(filter(lambda m: m >= 80, marks))
# Keep only distinctions (80+)
print(high_scorers)                             [82, 91, 88]

# filter() + len() – count passing students
pass_count = len(list(filter(lambda m: m >= 45, marks)))
# How many passed?
print(f'{pass_count} out of {len(marks)} students passed.')
# Output: 6 out of 8 students passed.
```

◆ reduce() — Collapse a Collection to One Value

```
from functools import reduce                    reduce() is in the functools module — must import it
```

```

numbers = [10, 20, 30, 40, 50]
total = reduce(lambda x, y: x + y, numbers)
print(total)

# Find the maximum using reduce
highest = reduce(lambda a, b: a if a > b else b, numbers)
print(highest)

```

Adds pairs: (10+20)=30, (30+30)=60, (60+40)=100, (100+50)=150
Output: 150

Keeps the larger of each pair — ends with the overall maximum
Output: 50

Function	Input	Output	Transforms or Filters?
map(fn, list)	list of n items	list of n items (each transformed)	Transforms every item — same count in and out
filter(fn, list)	list of n items	list of 0 to n items (subset)	Filters — only items where fn returns True survive
reduce(fn, list)	list of n items	one single value	Collapses — accumulates all items into one result

◆ Module 1 Capstone — Student Report Generator

This example uses everything from Module 1: a list of dictionaries (Sessions 9 and 11), a for loop (Session 5), conditional logic (Session 4), and functions (this session). Read it line by line and identify which session each part comes from.

```

# --- Functions ---
def get_grade(marks):
    if marks >= 90: return 'A+'
    elif marks >= 75: return 'A'
    elif marks >= 60: return 'B'
    elif marks >= 45: return 'C'
    else: return 'Fail'

def print_report(student):
    grade = get_grade(student['marks'])
    status = 'PASS' if student['marks'] >= 45 else 'FAIL'
    print(f"{'='*35}")
    print(f"  Name   : {student['name']}")
    print(f"  Marks  : {student['marks']}  Grade: {grade}  [{status}]")

```

Session 12 — pure function, takes marks, returns grade string

Session 4 — conditional

Session 12 — takes one dictionary, prints a formatted report

Calls another function — functions can call functions

Session 4 — ternary conditional

Session 11 — dict access

```
print(f" Course : {student.get('course', 'N/A')}")
```

Session 11 — safe get() with default

```
# --- Data (Session 9 + 11) ---
```

```
students = [
```

Session 9 — list of dicts

```
{'name': 'Riya Shah', 'marks': 88, 'course': 'Python'},
```

```
{'name': 'Arjun Patel', 'marks': 42, 'course': 'Django'},
```

```
{'name': 'Meera Dave', 'marks': 76},
```

No 'course' key — get() handles this gracefully

```
]
```

```
# --- Generate Reports (Session 5) ---
```

```
for student in students:
```

Session 5 — loop through every student

```
    print_report(student)
```

Session 12 — call the function once per student

```
# --- Summary with lambda + filter (Session 12) ---
```

```
pass_count = len(list(filter(lambda s: s['marks'] >= 45, students)))
```

Count passing students using filter + lambda

```
print(f'\nResult: {pass_count}/{len(students)} students passed.')
```

TRY THIS LIVE

Run the capstone code above. Then extend it in two ways: (1) Add a function called `class_average(students)` that takes the full list and returns the average marks across all students using a loop and a counter — then call it and print the result. (2) Use `map()` with a lambda to create a new list of just the grade strings for all students, then print it.

CASE STUDY — Razorpay Payment Fee Engine

Context — Razorpay, headquartered in Bengaluru, is India's leading payment gateway — processing payments for over 8 million businesses including Swiggy, Ola, BYJU'S, and thousands of small businesses across Gujarat. It handles over Rs 10 lakh crore in annual payment volume.

Challenge — Every payment processed carries a fee that depends on the payment method (UPI, card, netbanking, wallet), transaction amount, merchant category, and active offer codes. Calculating the fee for millions of daily transactions accurately — and being able to update the fee structure for any payment type without touching the rest of the system — was a core engineering requirement.

What they did — Each payment method has its own fee calculation function: `calculate_upi_fee(amount)`, `calculate_card_fee(amount, card_type)`, `calculate_netbanking_fee(amount, bank)`. A master `calculate_fee(amount, method, **kwargs)` function dispatches to the right one based on the method argument. When RBI changed UPI fee rules, only the UPI function was updated — the rest of the system continued unchanged.

Result — Razorpay processes over 450 crore transactions per year. Because fee logic lives in isolated, testable functions, the engineering team updated UPI pricing rules across the entire platform in under one hour when the regulation changed — with no downtime and no errors in other payment types.

Takeaway — Functions are how large systems stay maintainable. One rule, one function, one place to update — that is the principle that makes production code survivable.

Source: *Razorpay Engineering Blog, razorpay.com/blog; RBI Payment System Reports 2023–24*

“A function is a promise: give me the right inputs, and I will always give you the right output. That promise is the foundation of every reliable system ever built.”

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — Week 4, Lecture 1: Functions in Python (Prof. Madhavan Mukund, IIT Madras)

Link — <https://nptel.ac.in/courses/106106145>

MODULE 1 COMPLETE — What You Can Now Build

You have completed the 12 foundational sessions of Python. You can now write programs that: collect input from users and validate it (Sessions 3, 4), loop through collections automatically (Sessions 5, 6, 7), manipulate text with precision (Session 8), store and retrieve structured data as lists, tuples, and dictionaries (Sessions 9, 10, 11), and encapsulate logic into reusable, testable functions (Session 12). Module 2 goes deeper — advanced collection operations, custom modules, and the tools that turn these skills into real applications.

WHERE TO GO NEXT

You now have every foundational building block of Python. Every project from here — web apps, data pipelines, automation scripts — is built by combining these twelve sessions' worth of tools.

Explore deeper — Read the official Python tutorial on functions at docs.python.org/3/tutorial/controlflow.html#defining-functions — pay special attention to `*args` and `**kwargs` which allow functions to accept any number of arguments

Try on your own — Build a 'Mini Billing System': define functions for `calculate_gst(price, rate)`, `apply_discount(price, pct)`, and `generate_invoice(item_name, price, gst_rate, discount_pct)`. Call all three from a main loop that takes item details from the user and prints a formatted invoice.

Connect the dots — Module 2 starts with List Deep Dive — you will learn advanced patterns including list comprehensions, `zip()`, `sorted()` with custom keys, and nested data structures. Everything in Module 2 assumes fluency with Module 1.

Going further — Search 'Python `*args` and `**kwargs`' and 'Python decorators' — two advanced function features used constantly in Django, Flask, and every real Python framework you will encounter in Modules 3

and 4

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor for a beginner student in India. Explain in simple Hinglish the difference between a regular def function and a lambda, and when to use each. Show the same GST calculation written both ways — as a def function and as a lambda. Then show how the lambda version is used directly inside a map() call to apply GST to a list of five prices. Keep it under 200 words.

REVIEW



INTERVIEW PREP · MODULE 1

Interview Questions**◆ Fundamentals of Python Language**

Rehearse each answer out loud and aim to deliver it within 60 seconds. The hint is only a nudge — attempt the answer first, then compare with the model answer below.

1**What is Python, and why is it considered a popular programming language?**

Think about Python's readability, community, and typical uses.

MODEL ANSWER

Python is a high-level, interpreted programming language known for its clean, readable syntax that resembles plain English. It is popular because it has a massive standard library, a large active community, and works well across domains such as web development, data science, automation, and AI. Its low learning curve makes it a top choice for beginners, while its powerful ecosystem keeps professionals using it.

2**Give two real-world use cases where Python is commonly used.****MODEL ANSWER**

Python is widely used in data science and machine learning — libraries like NumPy, Pandas, and TensorFlow make it the dominant language in that field. It is also heavily used in web development, with frameworks like Django and Flask powering thousands of production websites and APIs.

3**How do you install Python on your computer and write your first script in VS Code?**

Mention both the installation process and the steps to create and run a .py file.

MODEL ANSWER

Download the Python installer from python.org and run it, making sure to check 'Add Python to PATH' during setup. Then install VS Code and the Python extension. Create a new file with a .py extension, write your code (e.g., `print('Hello')`), and run it from the VS Code terminal using the command `python filename.py`.

4**Explain how Python code is executed from writing to output.****MODEL ANSWER**

You write Python code in a .py file as plain text. When you run it, the Python interpreter reads your source code, converts it to bytecode, and the Python Virtual Machine (PVM) executes that bytecode line by line. The result is displayed in the console or terminal.

5

What are variables in Python, and how do you declare one?*Think about assignment and naming.***MODEL ANSWER**

A variable is a named container that stores a value in memory. In Python you declare a variable simply by writing a name, an equals sign, and a value — for example, `name = 'Ravi'`. Python determines the data type automatically based on the value assigned, so you do not need to declare a type explicitly.

6

List two rules you must follow when naming variables in Python.**MODEL ANSWER**

First, a variable name must start with a letter or an underscore, never a digit — so `age` is valid but `1age` is not. Second, variable names cannot use Python reserved keywords such as `if`, `for`, or `while`. Names are also case-sensitive, so `Score` and `score` are two different variables.

7

What is the difference between primitive and non-primitive data types in Python?*Focus on built-in simple types vs. more complex structures.***MODEL ANSWER**

Primitive data types are the basic building blocks: `int`, `float`, `bool`, and `str` — they hold a single value. Non-primitive types are more complex structures that can hold collections of values, such as `list`, `tuple`, `dictionary`, and `set`. Non-primitive types can store multiple items and support operations like iteration, indexing, and nesting.

8

Write Python code to create variables for a student profile containing name, age, and enrolled course.**MODEL ANSWER**

```
name = 'Anjali'
age = 20
course = 'Python Programming'
print(name, age, course)
```

Each variable holds a different data type: `name` and `course` are strings, while `age` is an integer. You access them simply by using the variable name.

9

What does the `type()` function return in Python? Give an example.**MODEL ANSWER**

The `type()` function returns the data type of the value or variable passed to it. For example, `type(42)` returns `<class 'int'>`, and `type('hello')` returns `<class 'str'>`. It is useful for debugging when you are unsure what kind of value a variable is holding.

10

How do you get input from a user in Python, and how do you display output?*Mention the relevant built-in functions.***MODEL ANSWER**

You use the built-in `input()` function to receive input — it always returns a string. To display output you use the `print()` function. For example: `name = input('Enter your name: ')` followed by `print('Hello,', name)` will greet the user by name.

11

How would you convert a string input to an integer in Python?**MODEL ANSWER**

Since `input()` always returns a string, you wrap it with `int()` to convert it. For example: `age = int(input('Enter your age: '))`. If the user enters a non-numeric value this will raise a `ValueError`, so in production code you would add error handling around it.

12

Write a Python program that takes two numbers from the user and prints their sum.*Remember to convert input to the correct type.***MODEL ANSWER**

```
a = int(input('Enter first number: '))
b = int(input('Enter second number: '))
print('Sum:', a + b)
```

Both inputs must be converted from strings to integers before adding them, otherwise Python would concatenate them as strings rather than perform arithmetic.

13

What is an escape character in Python strings? Give an example.**MODEL ANSWER**

An escape character is a backslash (`\`) followed by a character that together represent something special inside a string. For example, `\n` inserts a newline, `\t` inserts a tab, and `\\` inserts a literal backslash. So `print('Hello\nWorld')` prints 'Hello' and 'World' on separate lines.

14

Explain the difference between `if`, `elif`, and `else` statements in Python.**MODEL ANSWER**

The `if` statement evaluates a condition and runs its block only when that condition is `True`. The `elif` (else-if) lets you check additional conditions when the previous `if` was `False`, and you can chain multiple `elif` blocks. The `else` block runs only when all preceding `if/elif` conditions were `False`, acting as a catch-all default case.

15

Write a Python program that checks if a number entered by the user is positive, negative, or zero.

MODEL ANSWER

```
n = int(input('Enter a number: '))
if n > 0:
    print('Positive')
elif n < 0:
    print('Negative')
else:
    print('Zero')
```

The elif handles the negative case only when the first condition (positive) is False, and else covers the remaining case of exactly zero.

16

Describe how you would use nested if statements. Give a simple example.

MODEL ANSWER

A nested if is an if statement placed inside another if block, allowing you to check a second condition only after the first is already True. For example, to check if a person can drive: if age >= 18: if has_license: print('Can drive') else: print('Needs license'). This way the license check only runs when the age condition passes.

17

Write a Python program to determine if someone is eligible to donate blood. (Eligibility: age >= 18 and weight >= 50kg)

MODEL ANSWER

```
age = int(input('Age: '))
weight = float(input('Weight (kg): '))
if age >= 18 and weight >= 50:
    print('Eligible to donate')
else:
    print('Not eligible')
```

Using the 'and' operator means both conditions must be True simultaneously; if either fails, the person is not eligible.

18

What is a for loop in Python used for?

MODEL ANSWER

A for loop is used to iterate over a sequence — such as a list, string, range, or tuple — and execute a block of code once for each item. It is the preferred loop when you know in advance how many iterations are needed or when you want to process every element in a collection.

19

Write a for loop to print each fruit name from a list: ['apple', 'banana', 'cherry'].

MODEL ANSWER

```
fruits = ['apple', 'banana', 'cherry']
for fruit in fruits:
    print(fruit)
```

The variable fruit takes the value of each element in the list one at a time, so the loop prints 'apple', then 'banana', then 'cherry', each on its own line.

20

How can you use a for loop to count the number of vowels in a string?

MODEL ANSWER

```
text = input('Enter a string: ')
count = 0
for char in text.lower():
    if char in 'aeiou':
        count += 1
print('Vowels:', count)
```

The loop checks each character, and the membership test 'in' confirms whether it is a vowel. Converting to lowercase first ensures 'A' and 'a' are both counted.

21

What is the difference between a for loop and a while loop in Python?

Consider how each loop determines when to stop.

MODEL ANSWER

A for loop iterates over a known sequence or range and stops automatically when all items have been processed. A while loop runs as long as a given condition remains True, making it suitable when the number of iterations is not known in advance. If the while condition never becomes False, the loop runs forever, so you must ensure the condition eventually changes.

22

Write a while loop to print numbers from 1 to 5.

MODEL ANSWER

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

The counter i starts at 1 and increases by 1 each iteration. When i reaches 6, the condition i <= 5 becomes False and the loop exits.

23

How would you create a simple star pattern like this using a while loop? * ** ***

MODEL ANSWER

```
i = 1
while i <= 3:
    print('*' * i)
    i += 1
```

String multiplication ('*' * i) prints i stars on each line. The loop runs three times, incrementing i each time, producing rows of 1, 2, and 3 stars respectively.

24

What happens if the condition in a while loop never becomes False?

Think about infinite loops.

MODEL ANSWER

The loop runs indefinitely, creating an infinite loop that will hang your program. This happens when the loop variable is never updated or the condition can never become False. To stop an infinite loop you press Ctrl+C in the terminal. To prevent it, always make sure the loop body contains logic that eventually makes the condition False.

25

Explain the use of break, continue, and pass statements in loops.

MODEL ANSWER

break immediately exits the entire loop, regardless of the loop condition. continue skips the rest of the current iteration and jumps back to the loop's start to check the condition again. pass is a no-op placeholder that does nothing — it is used when syntax requires a statement but you do not want any action, such as in an empty block you plan to fill in later.

26

Write a for loop that prints each fruit in the list ['apple', 'banana', 'cherry'], but skips 'banana'.

MODEL ANSWER

```
for fruit in ['apple', 'banana', 'cherry']:
    if fruit == 'banana':
        continue
    print(fruit)
```

When fruit equals 'banana', continue skips the print statement and moves on to the next iteration, so only 'apple' and 'cherry' are printed.

27

How would you use `break` to exit a loop when a specific value is found?

MODEL ANSWER

```
numbers = [3, 7, 2, 9, 5]
for n in numbers:
    if n == 9:
        print('Found 9!')
        break
```

When 9 is found, `break` immediately terminates the loop so Python does not continue checking the remaining items, making the search more efficient.

28

Give an example where `pass` would be useful in a Python loop or function.

MODEL ANSWER

`Pass` is useful as a placeholder when you define a function or loop body but have not yet written the implementation. For example: `def process_data(): pass` — this is valid Python and lets you define the function structure first without causing a syntax error, then fill in the logic later.

29

How can you access the first and last character of a string in Python?

Think about string indexing.

MODEL ANSWER

Strings are indexed from 0 for the first character. So for `s = 'Python'`, `s[0]` gives 'P' (first character) and `s[-1]` gives 'n' (last character). Negative indexing counts from the end of the string, so `-1` is always the last element regardless of the string's length.

30

Write Python code to reverse a string entered by the user.

MODEL ANSWER

```
text = input('Enter a string: ')
print(text[::-1])
```

Slice notation `[::-1]` means start from the end and step backwards by 1, effectively reversing the string. This is the most Pythonic one-liner for string reversal.

31

What is string slicing? Give an example to extract a substring.

MODEL ANSWER

String slicing lets you extract a portion of a string using the syntax `s[start:end:step]`, where `start` is inclusive and `end` is exclusive. For example, `s = 'Hello World'`; `s[0:5]` gives 'Hello'. If you omit `start` it defaults to 0, and if you omit `end` it goes to the end of the string.

32

Name two string methods in Python and explain what they do.

MODEL ANSWER

`upper()` converts all characters in the string to uppercase — `'hello'.upper()` returns `'HELLO'`. `strip()` removes leading and trailing whitespace (or a specified character) from the string — `' hello '.strip()` returns `'hello'`. Both methods return a new string and do not modify the original.

33

How do you create a list in Python and add a new item to it?

MODEL ANSWER

A list is created using square brackets: `fruits = ['apple', 'mango']`. To add a new item to the end of the list you call the `append()` method: `fruits.append('banana')`. The list is mutable, meaning you can change, add, or remove items after creation.

34

Write code to remove the third element from a list.

MODEL ANSWER

```
items = [10, 20, 30, 40, 50]
items.pop(2) # removes 30 (index 2)
print(items) # [10, 20, 40, 50]
```

List indices start at 0, so index 2 is the third element. Alternatively, `del items[2]` also works. `pop()` additionally returns the removed value if you need to use it.

35

Explain the difference between the `append()` and `insert()` methods for lists.

MODEL ANSWER

`append()` always adds a single item to the end of the list — no position is specified. `insert()` lets you add an item at any chosen index: `insert(i, value)` places `value` at position `i`, shifting all existing elements after it one position to the right. Use `append()` for a simple end-of-list addition and `insert()` when position matters.

36

How would you sort a list of numbers in ascending order?

MODEL ANSWER

You can call the `sort()` method directly on the list — `nums.sort()` — which sorts it in place and modifies the original list. Alternatively, `sorted(nums)` returns a new sorted list without changing the original. Both sort in ascending order by default; pass `reverse=True` for descending order.

37

What is a tuple in Python, and how is it different from a list?

Focus on mutability.

MODEL ANSWER

A tuple is an ordered collection of items, just like a list, but it is immutable — once created, its elements cannot be changed, added, or removed. Tuples are defined with parentheses: `coords = (10, 20)`. Use a tuple when the data should remain constant throughout the program, such as for fixed coordinates or configuration values.

38

Write code to create a tuple with three elements and access its second value.

MODEL ANSWER

```
colors = ('red', 'green', 'blue')
print(colors[1]) # Output: green
```

Indexing works the same as with lists — zero-based. Since tuples are immutable you can read values by index but cannot reassign them.

39

How can you concatenate two tuples in Python?

MODEL ANSWER

You use the + operator to join two tuples into a new one: `t1 = (1, 2)`; `t2 = (3, 4)`; `t3 = t1 + t2` gives `(1, 2, 3, 4)`. This does not modify the original tuples; it creates a brand new tuple containing all elements of both.

40

What happens if you try to change an element of a tuple after creation?

MODEL ANSWER

Python raises a `TypeError`: 'tuple' object does not support item assignment. This is by design — tuples are immutable, meaning their contents are fixed after creation. If you need to modify the data, you should use a list instead, or convert the tuple to a list, make changes, and convert it back.

41

What is a dictionary in Python, and how do you add a new key-value pair?

MODEL ANSWER

A dictionary stores data as key-value pairs, where each key is unique and maps to a specific value: `student = {'name': 'Ravi', 'age': 20}`. To add a new pair you simply assign to a new key: `student['course'] = 'Python'`. If the key already exists, this overwrites its value; if it does not exist, it creates it.

42

Write code to update the value of an existing key in a dictionary.

MODEL ANSWER

```
student = {'name': 'Ravi', 'age': 20}
student['age'] = 21
print(student) # {'name': 'Ravi', 'age': 21}
```

Assigning to an existing key replaces its value. Alternatively, the `update()` method lets you update multiple keys at once: `student.update({'age': 21, 'course': 'Python'})`.

43

How can you get all the keys and all the values from a dictionary?

MODEL ANSWER

Use `dict.keys()` to return all keys and `dict.values()` to return all values. For example, `student.keys()` returns `dict_keys(['name', 'age'])`. To get both at once use `student.items()`, which returns key-value pairs as tuples — useful for iterating with a `for` loop.

44

Explain the difference between `keys()`, `values()`, and `items()` methods in dictionaries.

MODEL ANSWER

`keys()` returns a view of all the keys in the dictionary. `values()` returns a view of all the values. `items()` returns a view of all key-value pairs as tuples — for example, `[('name', 'Ravi'), ('age', 20)]`. All three return dynamic view objects that reflect changes to the dictionary, and they are typically used in for loops to iterate over the dictionary.

45

How do you define a function in Python that takes two numbers and returns their product?

MODEL ANSWER

```
def multiply(a, b):
    return a * b
```

```
result = multiply(4, 5) # result = 20
```

The `def` keyword defines the function, parameters are listed in parentheses, and `return` sends the result back to the caller. The function can be called multiple times with different values without rewriting the logic.

46

What is a lambda function? Write a lambda to square a number.

MODEL ANSWER

A lambda is an anonymous, single-expression function defined without the `def` keyword. The syntax is: `lambda arguments: expression`. For example, `square = lambda x: x ** 2`; `square(5)` returns 25. Lambdas are typically used for short, throwaway functions — especially as arguments to `map()`, `filter()`, or `sorted()`.

47

How would you use `map()` to double every number in a list?

MODEL ANSWER

```
numbers = [1, 2, 3, 4]
result = list(map(lambda x: x * 2, numbers))
print(result) # [2, 4, 6, 8]
```

`map()` applies the given function to every element of the iterable and returns a map object. Wrapping it in `list()` converts it to a regular list. It is a clean, functional alternative to writing a for loop.

48

Give an example of using `filter()` with a lambda to get only even numbers from a list.

MODEL ANSWER

```
numbers = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens) # [2, 4, 6]
```

`filter()` keeps only the elements for which the function returns `True`. The lambda checks divisibility by 2, so only even numbers pass through the filter.

49

How would you explain the concept of return values in Python functions?**MODEL ANSWER**

A return statement sends a value back from a function to the code that called it. Without a return, a function implicitly returns None. You can return any data type — a number, string, list, or even multiple values as a tuple. The caller can assign the returned value to a variable for further use, making functions composable and reusable.

50

Write a function that takes a list of numbers and returns the maximum value.**MODEL ANSWER**

```
def find_max(numbers):
    maximum = numbers[0]
    for n in numbers:
        if n > maximum:
            maximum = n
    return maximum

print(find_max([3, 7, 1, 9, 4])) # 9
```

The function initialises maximum with the first element and iterates, replacing it whenever a larger value is found. Python also has a built-in max() function, but writing it manually shows you understand the logic.

51

How would you use reduce() to calculate the sum of all elements in a list?**MODEL ANSWER**

```
from functools import reduce
numbers = [1, 2, 3, 4, 5]
total = reduce(lambda a, b: a + b, numbers)
print(total) # 15
```

reduce() takes a function and applies it cumulatively to the list: first it adds 1+2=3, then 3+3=6, then 6+4=10, then 10+5=15. It must be imported from the functools module in Python 3.

52

When would you use a function instead of repeating code in your program?**MODEL ANSWER**

Whenever you find yourself writing the same block of logic in more than one place, that is the signal to extract it into a function. Functions promote the DRY (Don't Repeat Yourself) principle — a bug fix or change needs to be made in only one place. They also make code easier to read, test, and reuse across different parts of the program or in other projects.

53

How could you use ChatGPT or GitHub Copilot to help you write or debug Python code, and what steps would you take to ensure the code it suggests is correct?

Think about testing and understanding suggestions.

MODEL ANSWER

You can paste an error message or describe the problem to ChatGPT and ask it to explain the issue and suggest a fix. GitHub Copilot suggests code as you type inside your editor. In both cases you should read and understand each suggestion before accepting it, run the code to verify it works on your actual data, and check edge cases manually. Never blindly paste AI-generated code into production without testing.

54

If you need to quickly generate example data or boilerplate code for your Python project, how might AI tools help, and what would you double-check before using their output?

MODEL ANSWER

AI tools can quickly generate sample datasets, class templates, function stubs, or repetitive boilerplate code that would take time to type manually. Before using their output you should verify that the variable names and logic match your project's structure, test the generated code with real inputs, and ensure it follows secure coding practices — especially if it touches files, databases, or user inputs.

55

Suppose you need to process a list of student records and extract names of students who scored above 80. Which combination of data structures and control flow statements from this module would you use, and why?

Consider lists, for loops, and conditionals together.

MODEL ANSWER

I would store student records as a list of dictionaries, where each dictionary holds a student's name and score. Then I would use a for loop to iterate through the list and an if statement to check whether the score key is greater than 80. The names passing the check would be appended to a separate results list. This approach is readable, extensible, and uses the core building blocks of lists, dicts, loops, and conditionals together.

56

How would you design a Python function that takes a string, checks if it is a palindrome, and returns True or False? Mention which string methods and control flow you would use.

MODEL ANSWER

```
def is_palindrome(s):  
    s = s.lower().replace(' ', '')  
    return s == s[::-1]
```

First, I use lower() to make the check case-insensitive and replace() to remove spaces. Then I compare the string with its reverse using slice [::-1]. If they match the string is a palindrome and the function returns True, otherwise False.

57

Imagine you are building a student management system. Which data type would you use to store each student's profile (name, age, courses), and how would you access or update their information efficiently?

Think about dictionaries and their key-value access.

MODEL ANSWER

I would use a dictionary for each student's profile, with keys like 'name', 'age', and 'courses' — where 'courses' could be a list. All students could then be stored in a list of these dictionaries or in a master dictionary keyed by student ID. Accessing data is fast with dict keys: `student['name']` is immediate. Updating is equally simple: `student['age'] = 21` or `student['courses'].append('Django')` adds a new course.